RV College of Engineering®

Artificial Intelligence and Machine learning

AI253IA

UNIT 3

Saraswathi G Datar

Assistant Professor,

Department of Computer Science and Engineering,

RVCE

Outline

- **Supervised Learning**
- **Decision Tree Classifier**
- **Model Overfitting**
- **Model Selection**

RV College of Engineering®

UNIT-3 Supervised Learning Basic Concepts, General Framework for Classification

Early History & Progress of Machine learning

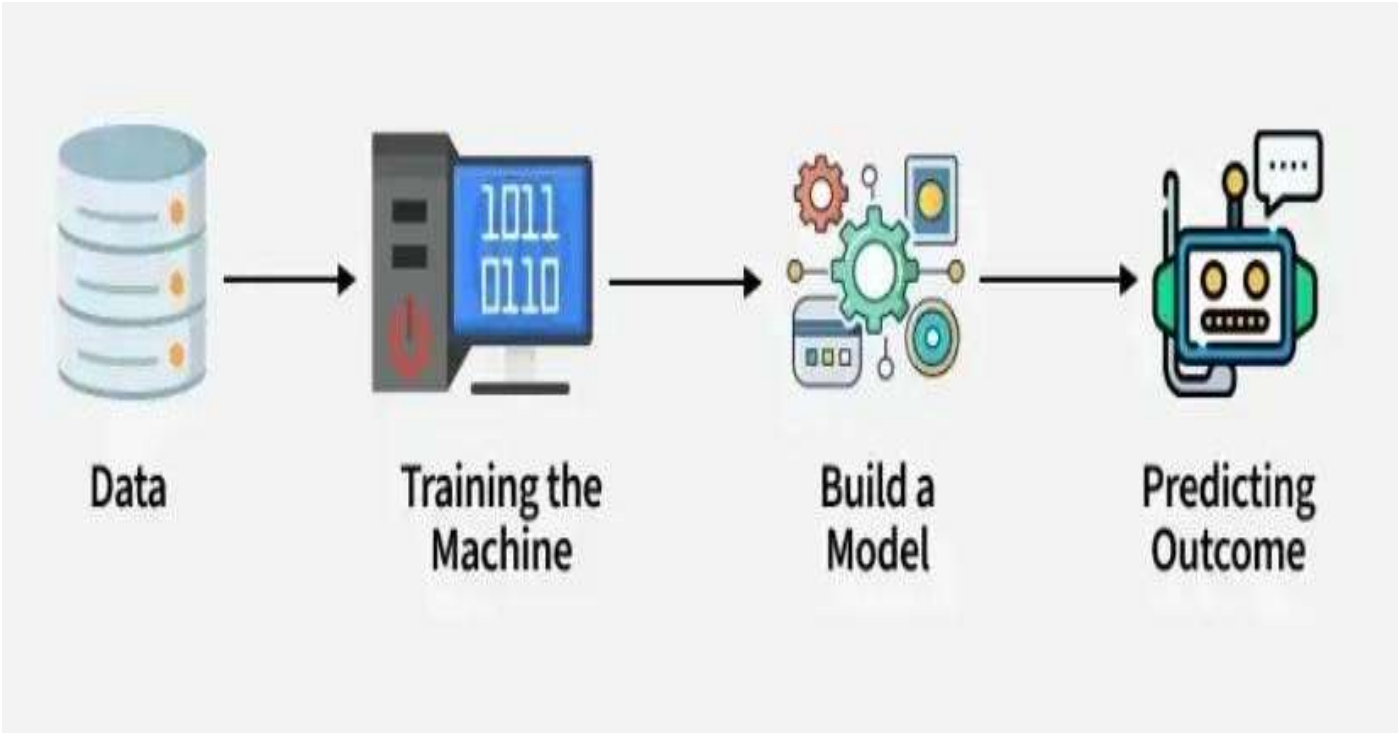
- Since computers were invented, researchers have asked: Can machines learn?
- Machines still cannot learn as well as humans- but progress is significant.
- Effective algorithms now exist for specific learning tasks.

Potential uses:

- Medical diagnosis & treatment recommendations
- Smart homes optimizing energy use
- Personal assistants adapting to user interests

What is Machine Learning?

- Machine Learning means teaching computers to learn from examples (data) and make predictions.



Well Posed Learning Problem

- **A computer program is said to learn from experience E with respect to some class of tasks *T* and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E.**

Clearly define three things:

- **Task (T)** — What the program should do.
- **Experience (E)** — How it learns (data, examples, practice).
- **Performance Measure (P)** — How you measure if it's getting better.

Well Posed Learning Problem

Example — Email Spam Filter

- **Task (T)**: Classify emails as spam or not spam.
- **Experience (E)**: The program learns from examples of emails labeled "spam" or "not spam."
- **Performance Measure (P)**: The percentage of emails correctly classified (accuracy).

Example : Robot Driving Learning Problem:

- **Task :** driving on public four-lane highways using vision sensors.
- **Performance measure :** average distance traveled before an error (as judged by human overseer)
- **Training experience :** a sequence of images and steering commands recorded while observing a human driver

Designing a Learning System

Choosing the Training Experience:

- Where will the learner get its experience or data? Will it get clear feedback or figure things out on its own?

Choosing the Target Function:

- What exactly do we want the learner to predict or decide? This is the goal or "rule" we want it to discover.

Choosing a Representation for the Target Function:

- How will the learner store or express what it has learned? (e.g., decision trees, rules, neural networks, etc.)

Choosing a Function Approximation Algorithm:

- What learning method or algorithm will we use to find the target function from the training examples?

Choosing the Training Experience

Direct vs Indirect Feedback

Direct Feedback:

- The program gets clear, exact guidance.
- *Example:* Someone tells the program the **best move** for each board position.
- *Easy to learn — the program knows exactly what's right or wrong.*

Indirect Feedback:

- The program only sees whether it **won or lost** the whole game.
- *It has to figure out **which moves were good or bad** by itself.*
- *Harder — because sometimes a good move might still lead to losing if later moves are bad.*

Choosing the Training Experience

Who Controls the Training Examples?.

- **Teacher-controlled:** A human teacher shows important situations and the best moves.
- **Learner-controlled (Self-play)**

Does the Training Look Like Real-World Testing?

Choosing the right training experience means:

- Give clear feedback if possible.
- Make sure the learner sees examples similar to what it will see in the real world.
- Decide who controls what it learns from — a teacher, the learner itself, or both.

Choosing the Target Function

- When you design a learning system, you must decide: What exactly do you want the program to learn to do? You want to build a computer program that predicts how much a house will sell for.
- **Task (T):** Predict house price
- **Experience (E):** Past data about houses — size, location, number of bedrooms, age, and the actual price they sold for.
- **Performance (P):** How close the predicted price is to the real selling price.
- **Direct Way: Predict the Exact Price**
- **The obvious goal is:**
- **Input:** Details about the house **Output:** Predicted price in dollars
- **Target function** is:

Price: H→R where H is the set of all possible houses (with features like size, location, etc.), and R means it outputs a real number (the price).

Choosing the Target Function

- **But what if direct prediction is too complex?**
- Suppose the housing market is huge and prices depend on complex factors- neighborhood trends, school ratings, upcoming development projects — some of which are hard to model directly.
- In that case, you might simplify the target function: Instead of predicting the exact price
- Predict a score: how desirable is the house?

Desirability→[0,100]

Choosing a representation for the Target Function

- The computer does not store knowledge the way humans do.
- It needs a clear, formal way to store, modify, and use what it has learned.

- **The representation you choose decides:**

1. How easy it is for the learner to learn it.
2. How well it works for real problems.
3. How fast predictions can be made.
4. Whether humans can understand it!

Choosing a representation for the Target Function

- **Spam Email Classifier**
  - **Target function:** Is this email spam or not? $f: E \rightarrow \{spam, not\ spam\}$
- **Set of Rules:**
- **"IF email contains 'win lottery' AND has an unknown sender THEN mark as spam."**
- **Decision Tree:**
- **A tree where you ask questions:**

Does the subject contain 'Congratulations'?

Does the sender have a trusted domain?

What is the probability of spam?

- **Linear Model:**
- **A mathematical formula like:**

Spam Score=2.3×(Has 'free')+1.1×(Contains link)−0.5×(Sender known)

Choosing a Function Approximation Algorithm

- Decide **what you want to learn** (the target function)
- Decide **how you'll represent it** (rules, decision tree, neural network, etc.)
- …the next big question is:
- **HOW will you actually *learn* it from the data?**
- This is where the **Function Approximation Algorithm** comes in.
- A **Function Approximation Algorithm** is the *learning method* that adjusts your chosen representation to fit the training data.
- If your representation is a decision tree → you need a tree-learning algorithm.
- If your representation is a neural network → you need an algorithm that adjusts the network's weights.
- If it's a line or curve → you need an algorithm that finds the best line or curve

Adjusting the Weights

$$E = \sum (true - predicted)^2$$

For each training example, you know:

- The true value (from training data)
- The predicted value (from your model)

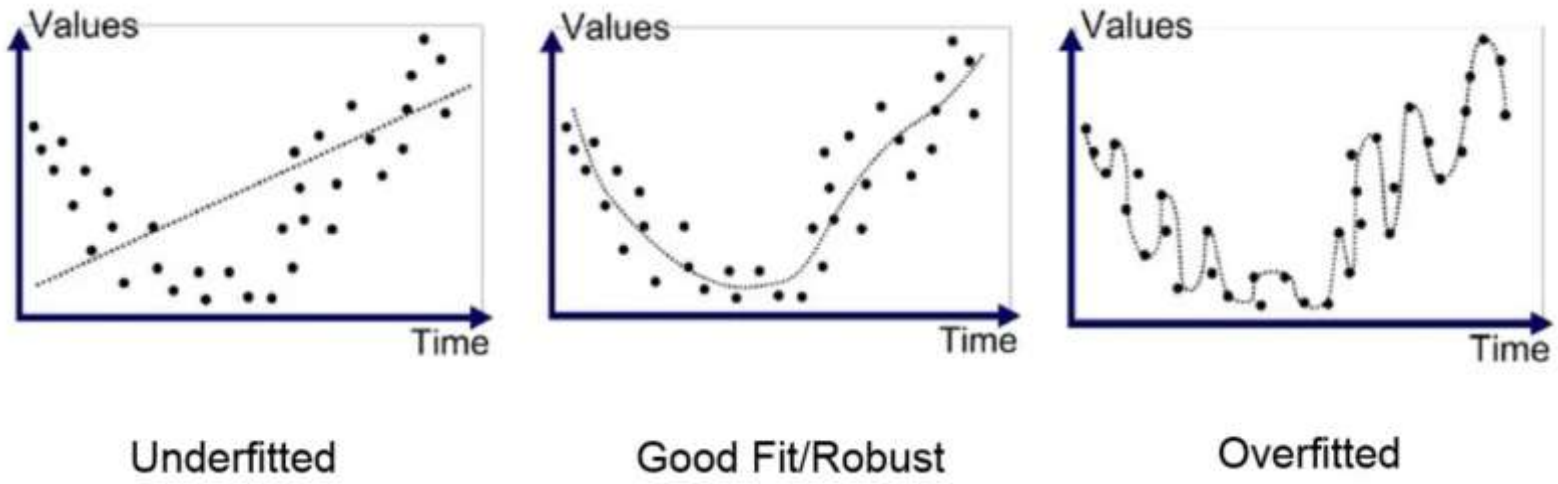
How ML works

1. Collect Data: Gather examples (numbers, text, images, etc.)
2. Train Model: Show examples to the computer to learn patterns.
3. Test Model: Check learning using new examples.
4. Use Model: Apply it to real-life predictions.

- Example: Like teaching a child: show many apples and bananas, test with a new fruit.
- Input Data = The examples you give (like pictures of fruits, sounds, or text).
- Labels = The correct answers for each example (like “apple” or “banana”).
- Learning = The machine looks at the data again and again to find patterns.
- Prediction = After learning, it tries to guess new examples by applying what it learned.

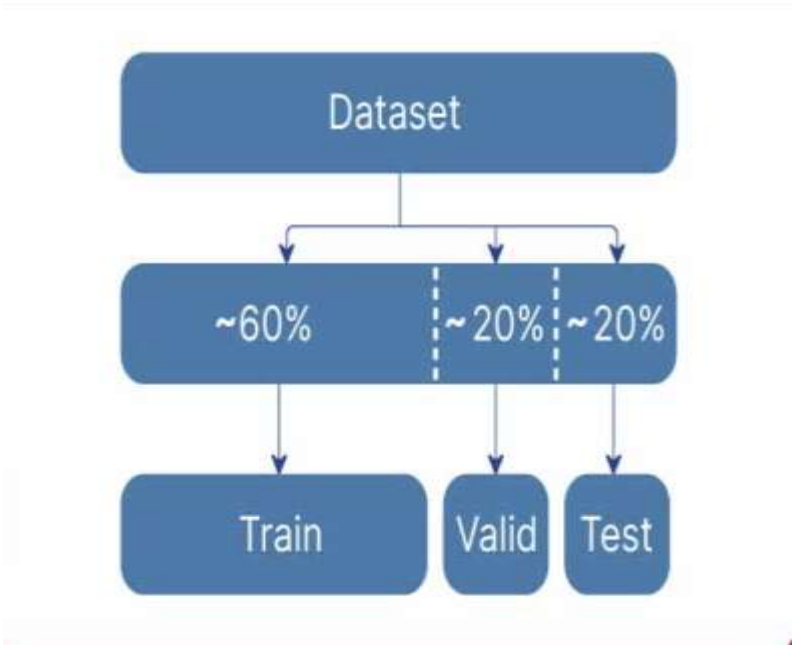
How ML works

- **Overfitting:**
- **A teacher teaches only from the last 5 years' question papers.**
- **Students do well if the same questions repeat — fail badly if even slightly new ones come.**
- **Underfitting:**
- **The teacher is vague, just talks theory with no examples or practice.**
- **Students don't learn how to answer any questions at all.**



What is Train-Test Split

- **We divide data into two parts:**
- **Training Set (80%): Used to teach the model.**
- **Test Set (20%): Used to check learning.**
- **Example: Study 8 chapters (training), exam from 2 new chapters (testing).**



- **If we train and test on same data, model may memorize answers.**
- **We want the model to understand patterns, not memorize.**
- **Example: Reading answers before exam vs truly understanding the topic.**

Evaluation Metrics(Judging the Model)

- Accuracy: How many predictions were correct?
- Precision: Out of predicted 'YES', how many were actually 'YES'?
- Recall: Out of actual 'YES', how many were caught?
- F1 Score: Balance between Precision & Recall

CONFUSION MATRIX

TP = True Positives
TN = True Negatives
FP = False Positives
FN = False Negatives

	p' (Predicted)	n' (Predicted)
p (Actual)	True Positive	False Negative
n (Actual)	False Positive	True Negative

$$\textbf{Precision} = \frac{TP}{TP + FP}$$

$$\textbf{Recall} = \frac{TP}{TP + FN}$$

$$\textbf{F1-score} = \frac{2 * \text{precision} * \text{recall}}{\text{precision} + \text{recall}}$$

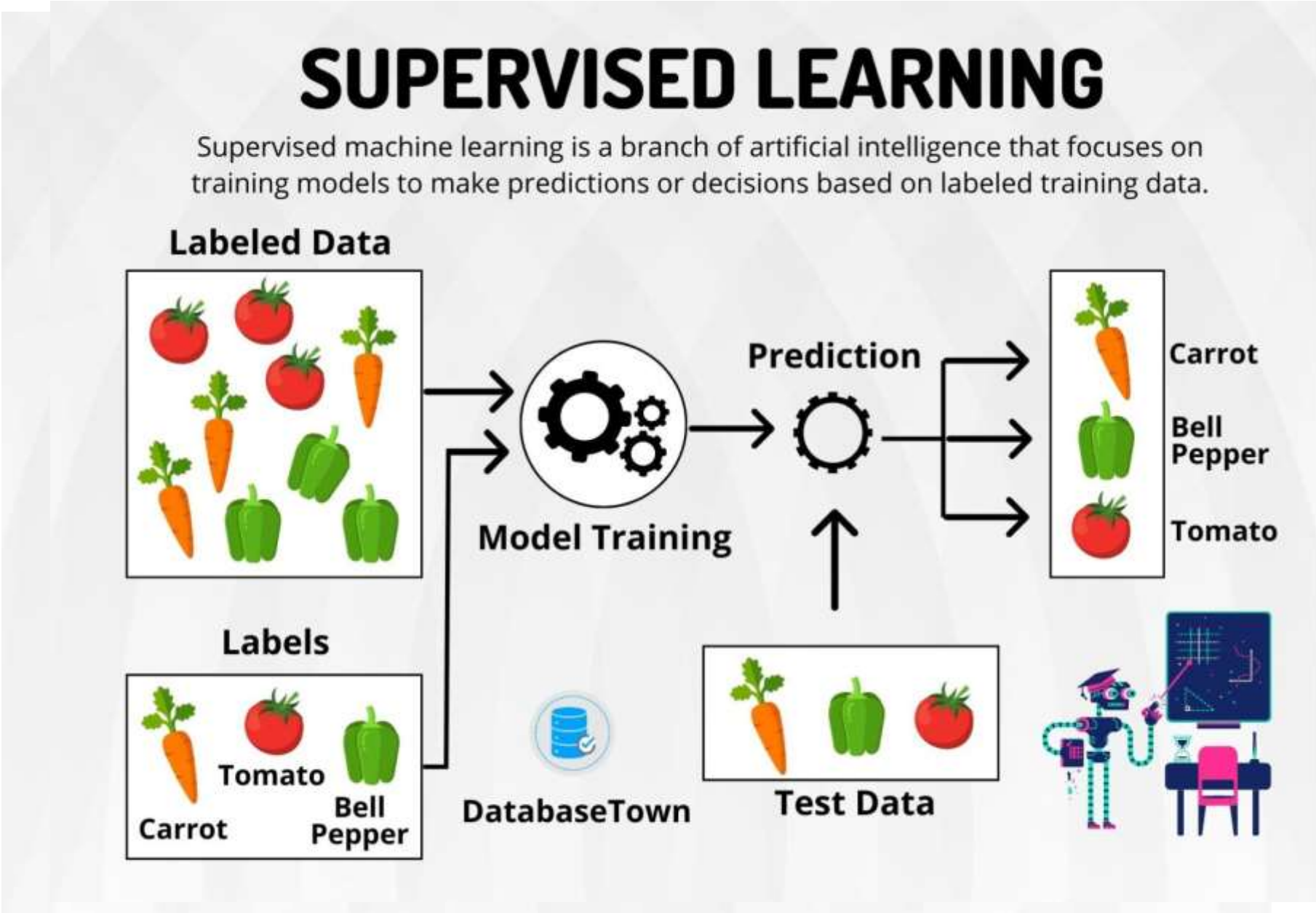
Summary of ML Process

- DATA → TRAINING → MODEL → TESTING → EVALUATION → DEPLOYMENT

- Collect Data
- Train Model
- Test Model
- Evaluate Results
- Deploy Model if results are good.

SUPERVISED LEARNING

Introduction



Introduction

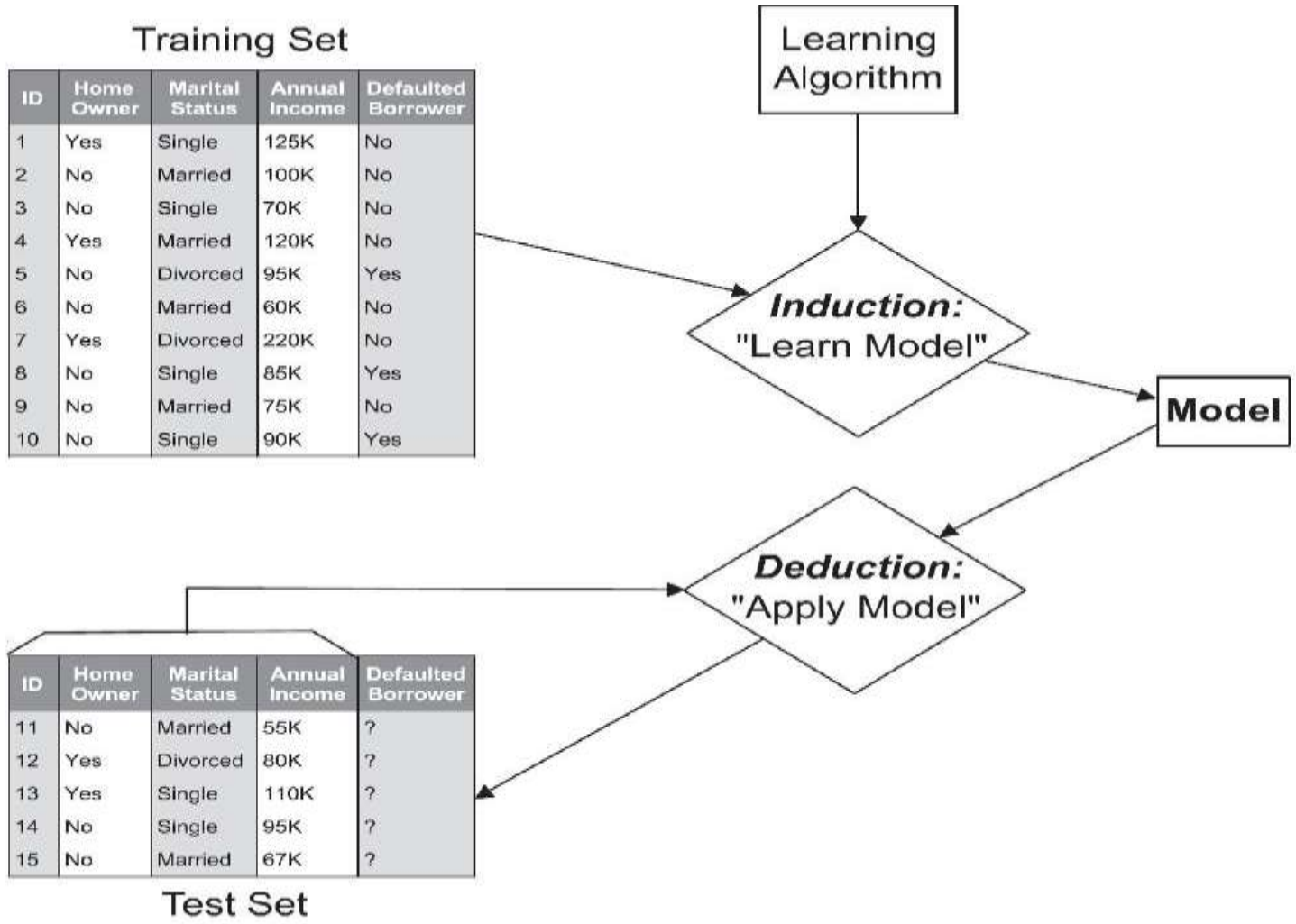
- Given a collection of records (training set)
  - Each record is by characterized by a tuple ($\boldsymbol{x}$,y), where $\boldsymbol{x}$ is the attribute set and y is the class label
    - $\boldsymbol{x}$: attribute, predictor, independent variable, input
    - y: class, response, dependent variable, output

- Task:
  - Learn a model that maps each attribute set $\boldsymbol{x}$ into one of the predefined class labels y

Introduction

Task	Attribute set, $\boldsymbol{x}$	Class label, y
Categorizing email messages	Features extracted from email message header and content	spam or non-spam
Identifying tumor cells	Features extracted from x-rays or MRI scans	malignant or benign cells
Cataloging galaxies	Features extracted from telescope images	Elliptical, spiral, or irregular-shaped galaxies

General Framework For Classification



ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
11	No	Married	55K	?
12	Yes	Divorced	80K	?
13	Yes	Single	110K	?
14	No	Single	95K	?
15	No	Married	67K	?

Figure 3.3. General framework for building a classification model.

General Framework For Classification

Table 3.4. Confusion matrix for a binary classification problem.

		Predicted Class	
		Class = 1	Class = 0
Actual Class	Class = 1	f_{11}	f_{10}
	Class = 0	f_{01}	f_{00}

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total number of predictions}}.$$

$$\text{Accuracy} = \frac{f_{11} + f_{00}}{f_{11} + f_{10} + f_{01} + f_{00}}.$$

$$\text{Error rate} = \frac{\text{Number of wrong predictions}}{\text{Total number of predictions}} = \frac{f_{10} + f_{01}}{f_{11} + f_{10} + f_{01} + f_{00}}.$$

Classification Techniques

- Base Classifiers
  - **Decision Tree based Methods**
  - Rule-based Methods
  - **Nearest-neighbor**
  - **Naïve Bayes** and Bayesian Belief Networks
  - Support Vector Machines
  - Neural Networks, Deep Neural Nets

- Ensemble Classifiers
  - Boosting, Bagging, Random Forests

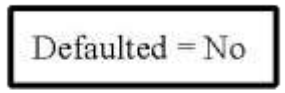
Decision Tree Induction

- Many Algorithms:
  - Hunt's Algorithm (one of the earliest)
  - CART
  - ID3, C4.5
  - SLIQ,SPRINT

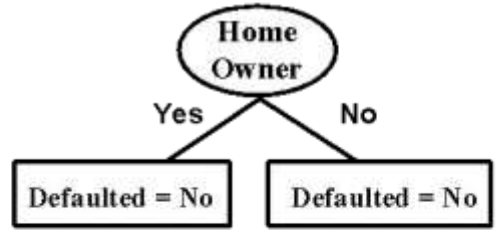
A Basic algorithm to Build a Decision Tree

- Hunt's algorithm is one of the earliest methods for constructing decision trees. It laid the groundwork for widely used classifiers like **ID3**, **C4.5**, and **CART**. The algorithm builds a decision tree recursively by partitioning the training data based on attribute tests, aiming to separate instances by class labels.
- Let D_t be the set of training records that reach a node t.
- General Procedure:
  - If D_t contains records that belong the same class y_t, then t is a leaf node labeled as y_t
  - If D_t contains records that belong to more than one class, use an attribute test to split the data into smaller subsets. Recursively apply the procedure to each subset.

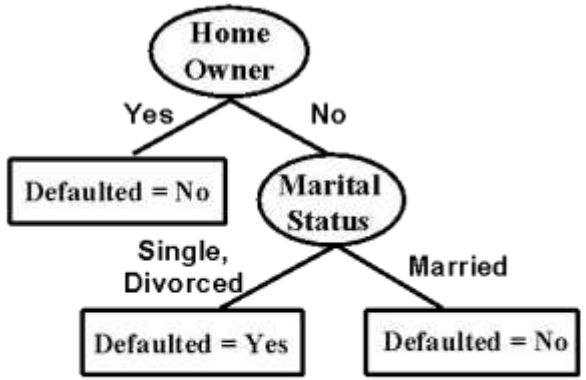
Hunt's Algorithm



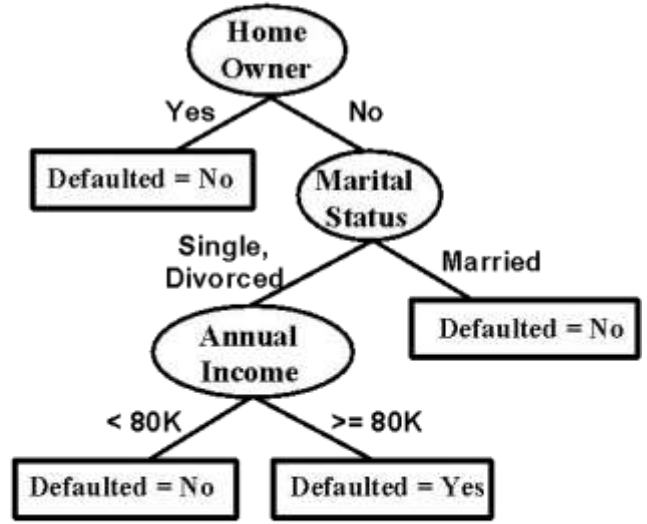
(a)



(b)



(c)



(d)

ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Design Issues of Decision Tree Induction

- How should training records be split?
  - Method for expressing test condition
    - depending on attribute types
  - Measure for evaluating the goodness of a test condition

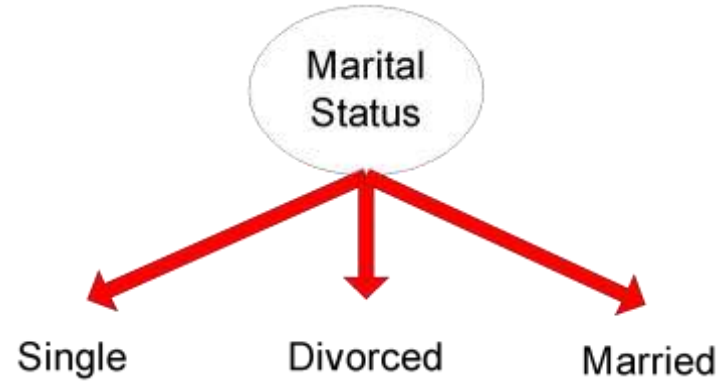
- How should the splitting procedure stop?
  - Stop splitting if all the records belong to the same class or have identical attribute values
  - Early termination

Methods for Expressing Test Conditions

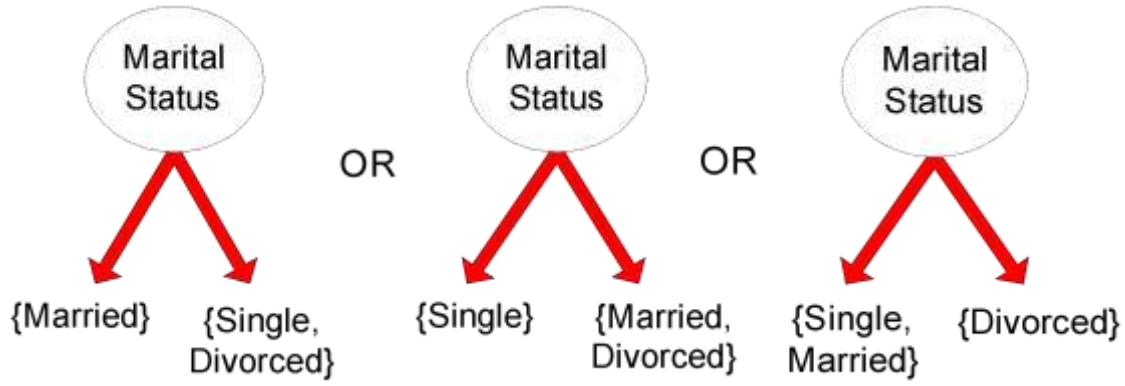
- Depends on attribute types
  - Binary
  - Nominal
  - Ordinal
  - Continuous

Test Conditions for Nominal Attributes

- Multi-way split:
  - Use as many partitions as distinct values.

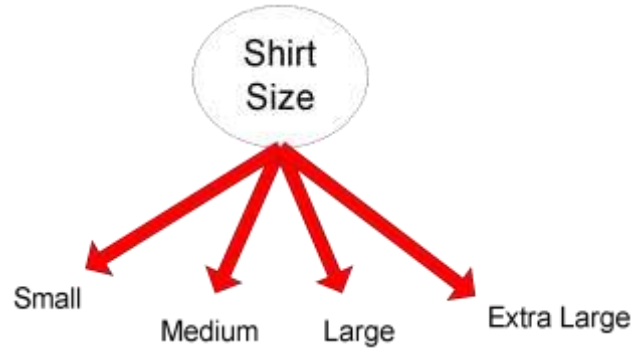


- Binary split:
  - Divides values into two subsets

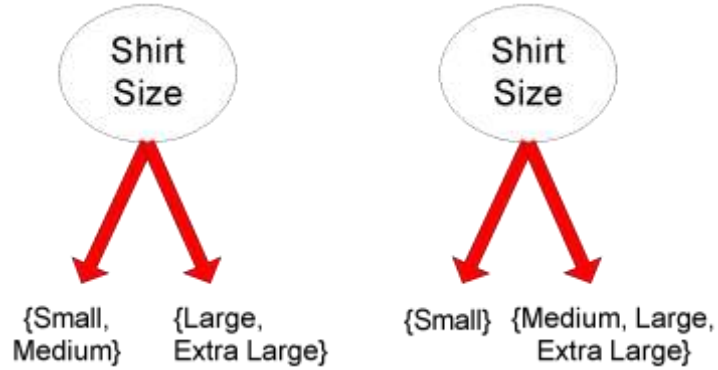


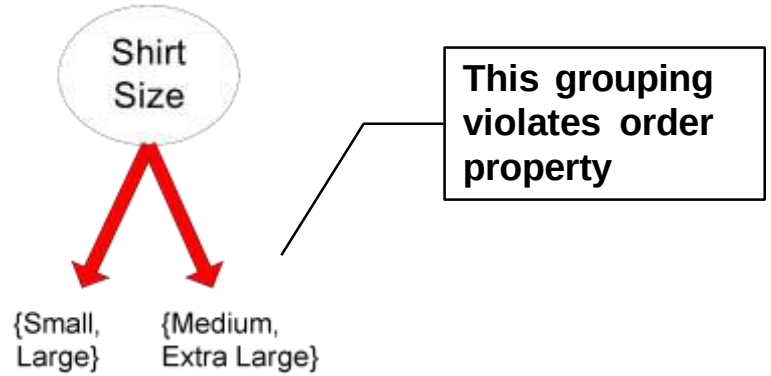
Test Conditions for Ordinal Attributes

- Multi-way split:
  - Use as many partitions as distinct values

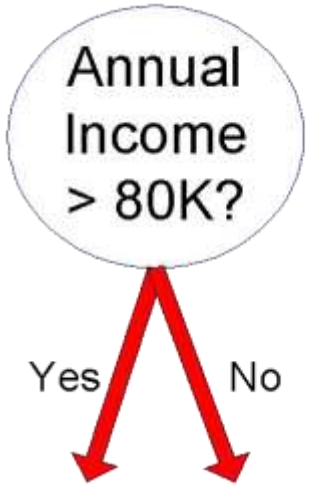


- Binary split:
  - Divides values into two subsets
  - Preserve order property among attribute values

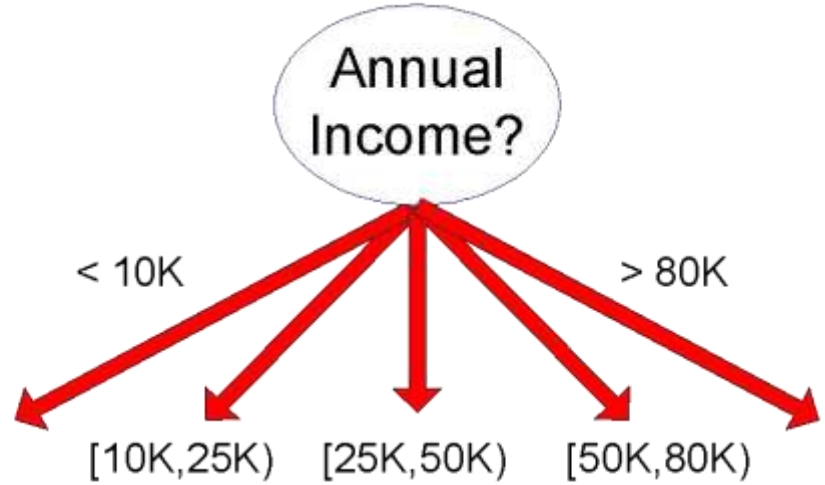




Test Conditions for Continuous Attributes

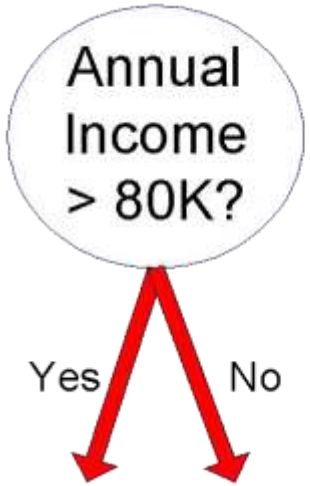


(i) Binary split

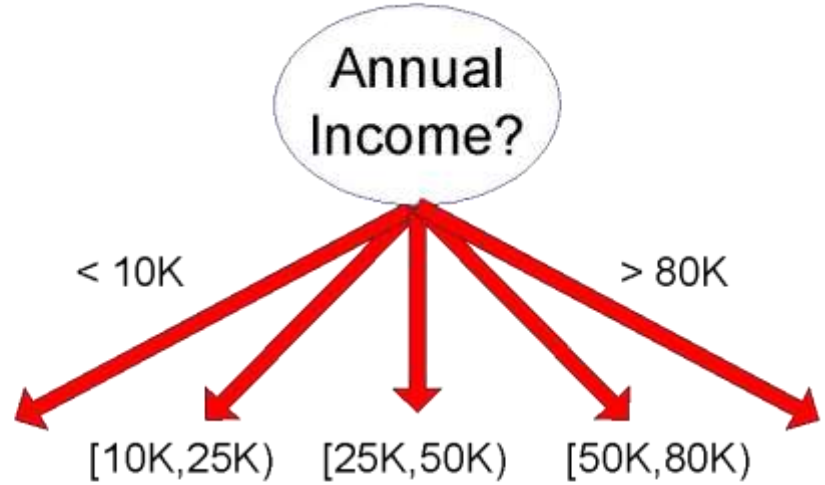


(ii) Multi-way split

Test Conditions for Continuous Attributes



(i) Binary split



(ii) Multi-way split

Measures for Selecting an Attribute Test Condition

Measures of Node Impurity

- Gini Index

$$Gini\ Index = 1 - \sum_{i=0}^{c-1} p_i(t)^2$$

Where $\boldsymbol{p_i(t)}$ is the frequency of class $\boldsymbol{i}$ at node **t**, and $\boldsymbol{c}$ is the total number of classes

- Entropy

$$Entropy = -\sum_{i=0}^{c-1} p_i(t) log_2 p_i(t)$$

- Misclassification error

$$Classification\ error = 1 - \max[p_i(t)]$$

Gini Index

- Gini Index for a given node t

$$Gini\ Index = 1 - \sum_{i=0}^{c-1} p_i(t)^2$$

Gini Index is probability of misclassifying a randomly chosen element in a set.

Where $p_i(t)$ is the frequency of class i at node t, and c is the total number of classes

  - Maximum of $1 - 1/c$ when records are equally distributed among all classes, implying the least beneficial situation for classification
  - Minimum of 0 when all records belong to one class, implying the most beneficial situation for classification

- For 2-class problem (p, 1 – p):
  - GINI = $1 - p^2 - (1 - p)^2 = 2p\,(1-p)$

C1	**0**
C2	**6**
Gini=0.000	

C1	**1**
C2	**5**
Gini=0.278	

C1	**2**
C2	**4**
Gini=0.444	

C1	**3**
C2	**3**
Gini=0.500	

Gini Index

- Single Node:

$$Gini\ Index = 1 - \sum_{i=0}^{c-1} p_i(t)^2$$

C1	**0**
C2	**6**

P(C1) = 0/6 = 0 P(C2) = 6/6 = 1

Gini = 1 – P(C1)^2 – P(C2)^2 = 1 – 0 – 1 = 0

C1	**1**
C2	**5**

P(C1) = 1/6 P(C2) = 5/6

Gini = 1 – (1/6)^2 – (5/6)^2 = 0.278

C1	**2**
C2	**4**

P(C1) = 2/6 P(C2) = 4/6

Gini = 1 – (2/6)^2 – (4/6)^2 = 0.444

Gini Index

- Gini Index for a collection of Nodes:

  - When a node p is split into k partitions (children)

$$GINI_{split} = \sum_{i=1}^{k} \frac{n_i}{n} GINI(i)$$

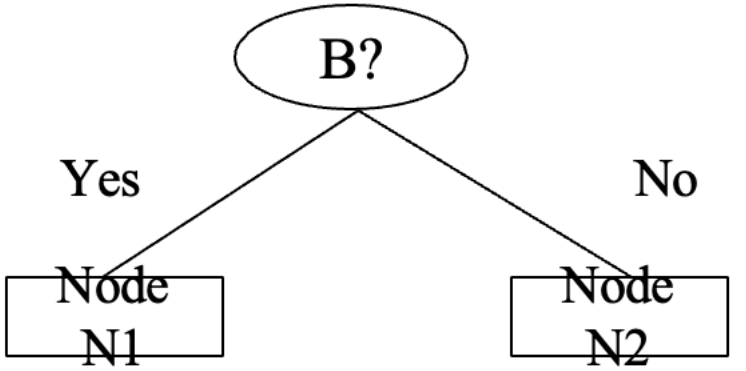
where, n_i = number of records at child i,

n = number of records at parent node p.

Binary Attributes: Computing Gini Index

- Splits into two partitions (child nodes)
- Effect of Weighing partitions:
  - Larger and purer partitions are sought

	Parent
C1	7
C2	5
Gini = 0.486	



	N1	N2
C1	5	2
C2	1	4
Gini=0.361		

Gini(N1)
= 1 – $(5/6)^2$ – $(1/6)^2$
= 0.278

Gini(N2)
= 1 – $(2/6)^2$ – $(4/6)^2$
= 0.444

Weighted Gini of N1 N2
= 6/12 * 0.278 +
6/12 * 0.444
= 0.361

Gain = 0.486 – 0.361 = 0.125

Categorical Attributes: Computing Gini Index

- For each distinct value, gather counts for each class in the dataset
- Use the count matrix to make decisions

Multi-way split

	CarType		
	Family	Sports	Luxury
C1	1	8	1
C2	3	0	7
Gini	0.163		

Two-way split
(find best partition of values)

	CarType	
	{Sports, Luxury}	{Family}
C1	9	1
C2	7	3
Gini	0.468	

	CarType	
	{Sports}	{Family, Luxury}
C1	8	2
C2	0	10
Gini	0.167	

Which of these is the best?

Continuous Attributes: Computing Gini Index

- Use Binary Decisions based on one value
- Several Choices for the splitting value
  - Number of possible splitting values = Number of distinct values
- Each splitting value has a count matrix associated with it
  - Class counts in each of the partitions, A ≤ v and A > v
- Simple method to choose best v
  - For each v, scan the database to gather count matrix and compute its Gini index
  - Computationally Inefficient! Repetition of work.

ID	Home Owner	Marital Status	Annual Income	Defaulted
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Annual Income ?

	≤ 80	> 80
Defaulted Yes	0	3
Defaulted No	3	4

Continuous Attributes: Computing Gini Index

- For efficient puting
  - Sort the attribu
  - Linearly scan t gini index
  - Choose the sp

$$Split\ Point = 80$$

$$Gini(< 80) = 1 - \sum_{i=1}^{c}(p_i)^2$$

$$= 1 - \left(\frac{0}{3}\right)^2 - \left(\frac{3}{3}\right)^2 = 0.0$$

$$Gini(\geq 80) = 1 - \sum_{i=1}^{c}(p_i)^2$$

$$= 1 - \left(\frac{3}{7}\right)^2 - \left(\frac{4}{7}\right)^2 = 0.4897$$

$$Gini(80) = w_1 * Gini(< 80) + w_2 * Gini(\geq 80)$$

$$= \frac{3}{10} * 0.0 + \frac{7}{10} * 0.4897 = 0.3427$$

Cheat	No		No		No		Yes		Yes		Yes		No		No		No		No		No	
	Annual Income																					
Sorted Values →	60		70		75		85		90		95		100		120		125		220			
Split Positions	55		65		72		80		87		92		97		110		122		172		230	
	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>
Yes	0	3	0	3	0	3	0	3	1	2	2	1	3	0	3	0	3	0	3	0	3	0
No	0	7	1	6	2	5	3	4	3	4	3	4	3	4	4	3	5	2	6	1	7	0
Gini	0.420		0.400		0.375		0.343		0.417		0.400		*0.300*		0.343		0.375		0.400		0.420	

Continuous Attributes: Computing Gini Index

Cheat	No	No	No	Yes	Yes	Yes	No	No	No	No
	Annual Income									
	60	70	75	85	90	95	100	120	125	220

	55		65		72		80		87		92		97		110		122		172		230	
	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>
Yes	0	3	0	3	0	3	0	3	1	2	2	1	3	0	3	0	3	0	3	0	3	0
No	0	7	1	6	2	5	3	4	3	4	3	4	3	4	4	3	5	2	6	1	7	0
Gini	0.420		0.400		0.375		0.343		0.417		0.400		*0.300*		0.343		0.375		0.400		0.420	

$$Split\ Point = 80$$

$$Gini(< 80) = 1 - \sum_{i=1}^{c}(p_i)^2$$

$$= 1 - \left(\frac{0}{3}\right)^2 - \left(\frac{3}{3}\right)^2 = 0.0$$

$$Gini(\geq 80) = 1 - \sum_{i=1}^{c}(p_i)^2$$

$$= 1 - \left(\frac{3}{7}\right)^2 - \left(\frac{4}{7}\right)^2 = 0.4897$$

$$Gini(80) = w_1 * Gini(< 80) + w_2 * Gini(\geq 80)$$

$$= \frac{3}{10} * 0.0 + \frac{7}{10} * 0.4897 = 0.3427$$

Measure of Impurity:Entropy

- Entropy at a given node $\boldsymbol{t}$

$$Entropy = -\sum_{i=0}^{c-1} p_i(t) log_2 p_i(t)$$

Where $\boldsymbol{p_i(t)}$ is the frequency of class $\boldsymbol{i}$ at node $\boldsymbol{t}$, and $\boldsymbol{c}$ is the total number of classes

  - Maximum of $\log_2 c$ when records are equally distributed among all classes, implying the least beneficial situation for classification
  - Minimum of 0 when all records belong to one class, implying most beneficial situation for classification

– Entropy based computations are quite similar to the GINI index computations

Entropy for a Single Node

$$Entropy = -\sum_{i=0}^{c-1} p_i(t) log_2 p_i(t)$$

C1	**0**
C2	**6**

P(C1) = 0/6 = 0 P(C2) = 6/6 = 1

Entropy = – 0 log 0 – 1 log 1 = – 0 – 0 = 0

C1	**1**
C2	**5**

P(C1) = 1/6 P(C2) = 5/6

Entropy = – (1/6) $\log_2$ (1/6) – (5/6) $\log_2$ (1/6) = 0.65

C1	**2**
C2	**4**

P(C1) = 2/6 P(C2) = 4/6

Entropy = – (2/6) $\log_2$ (2/6) – (4/6) $\log_2$ (4/6) = 0.92

Computing Information Gain After Splitting

- Information Gain:

$$Gain_{split} = Entropy(p) - \sum_{i=1}^{k} \frac{n_i}{n} Entropy(i)$$

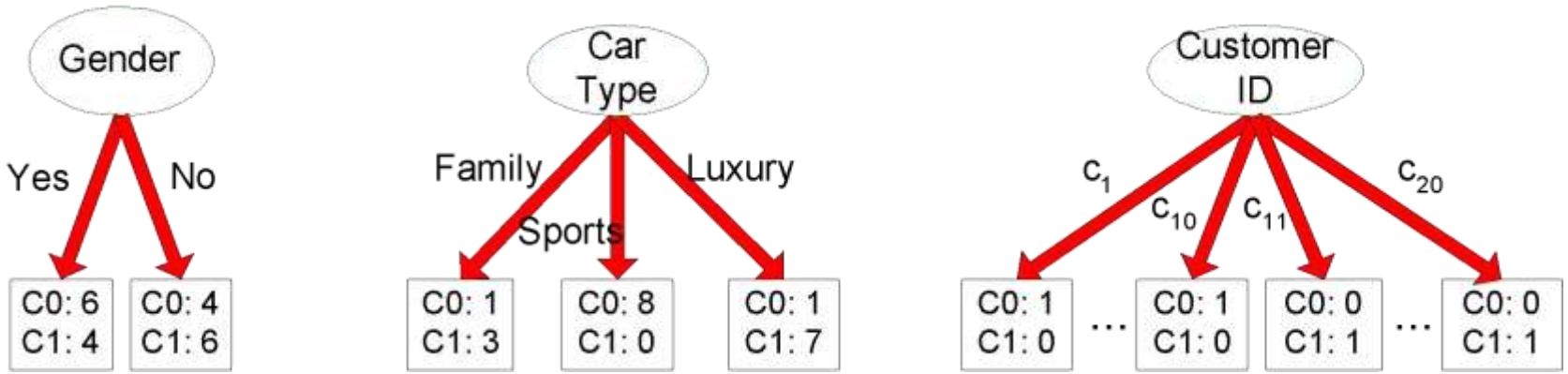
Parent Node, p is split into k partitions (children)

n_i is number of records in child node i

  - Choose the split that achieves most reduction (maximizes GAIN)
  - Used in the ID3 and C4.5 decision tree algorithms
  - Information gain is the mutual information between the class variable and the splitting variable

Problem with Large number of Partitions

- Node impurity measures tend to prefer splits that result in large number of partitions, each being small but pure.
- **High-cardinality attributes and complex trees**



  - Customer ID has highest information gain because entropy for all the children is zero

Gain Ratio

- Gain Ratio:

$$Gain\ Ratio = \frac{Gain_{split}}{Split\ Info} \qquad Split\ Info = -\sum_{i=1}^{k} \frac{n_i}{n} log_2 \frac{n_i}{n}$$

Parent Node, p is split into k partitions (children)

n_i is number of records in child node i

- Adjusts Information Gain by the entropy of the partitioning ($Split\ Info$).
  - Higher entropy partitioning (large number of small partitions) is penalized!
- Used in C4.5 algorithm
- Designed to overcome the disadvantage of Information Gain

Gain Ratio

- Gain Ratio:

$$Gain\ Ratio = \frac{Gain_{split}}{Split\ Info} \qquad Split\ Info = \sum_{i=1}^{k} \frac{n_i}{n} log_2 \frac{n_i}{n}$$

Parent Node, p is split into k partitions (children)

n_i is number of records in child node i

	CarType		
	Family	Sports	Luxury
C1	1	8	1
C2	3	0	7
Gini	0.163		

	CarType	
	{Sports, Luxury}	{Family}
C1	9	1
C2	7	3
Gini	0.468	

	CarType	
	{Sports}	{Family, Luxury}
C1	8	2
C2	0	10
Gini	0.167	

Classification Error

- Classification error at a node t

Talks about model's incorrect predictions

$$Error(t) = 1 - \max_i [p_i(t)]$$

  - Maximum of $1 - 1/c$ when records are equally distributed among all classes, implying the least interesting situation
  - Minimum of 0 when all records belong to one class, implying the most interesting situation

Classification Error for a Single Node

$$Error(t) = 1 - \max_i [p_i(t)]$$

C1	**0**
C2	**6**

P(C1) = 0/6 = 0 P(C2) = 6/6 = 1

Error = 1 – max (0, 1) = 1 – 1 = 0

C1	**1**
C2	**5**

P(C1) = 1/6 P(C2) = 5/6

Error = 1 – max (1/6, 5/6) = 1 – 5/6 = 1/6

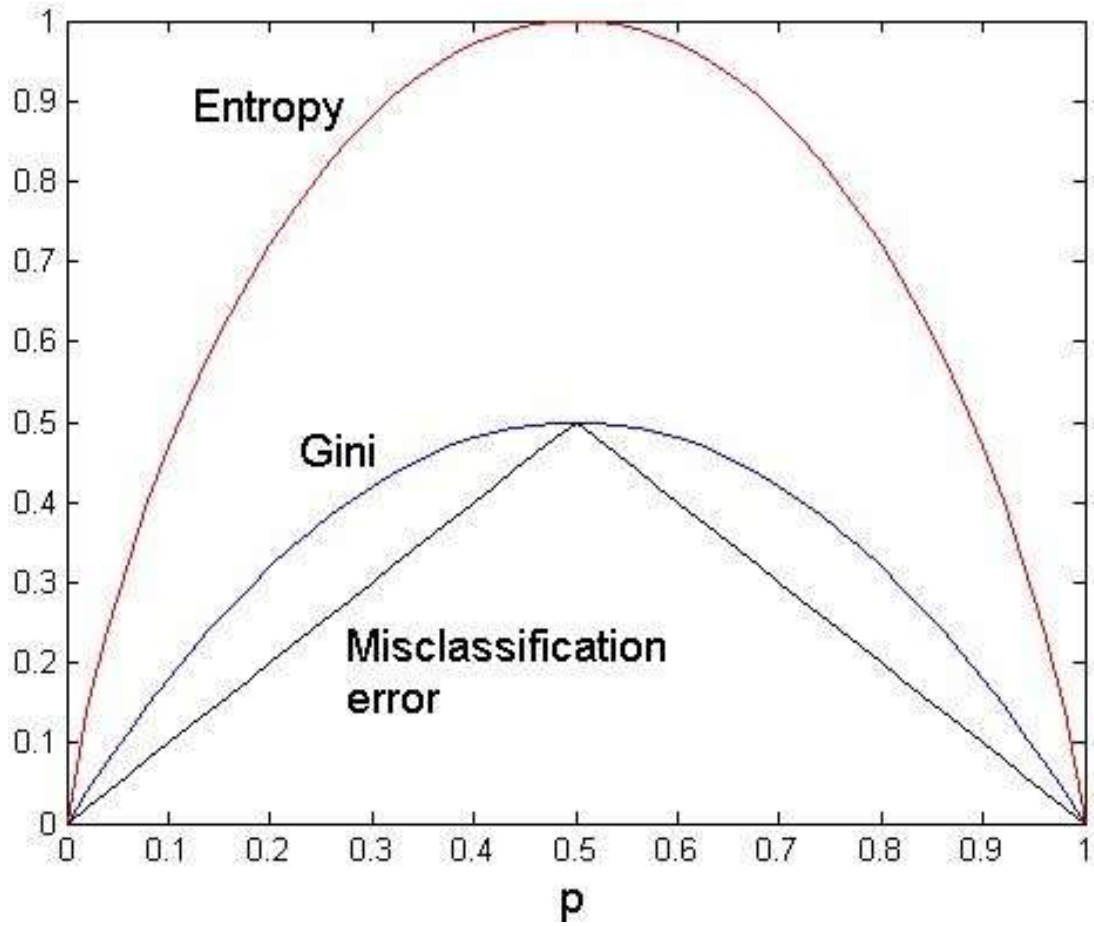
C1	**2**
C2	**4**

P(C1) = 2/6 P(C2) = 4/6

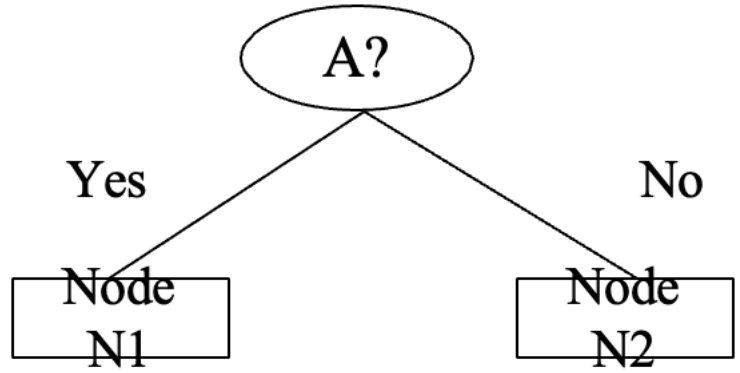
Error = 1 – max (2/6, 4/6) = 1 – 4/6 = 1/3

Measures

For a 2-class problem:



Classification Error vs Gini Index



	Parent
C1	7
C2	3
Gini = 0.42	

Gini(N1)
= 1 – $(3/3)^2$ – $(0/3)^2$
= 0

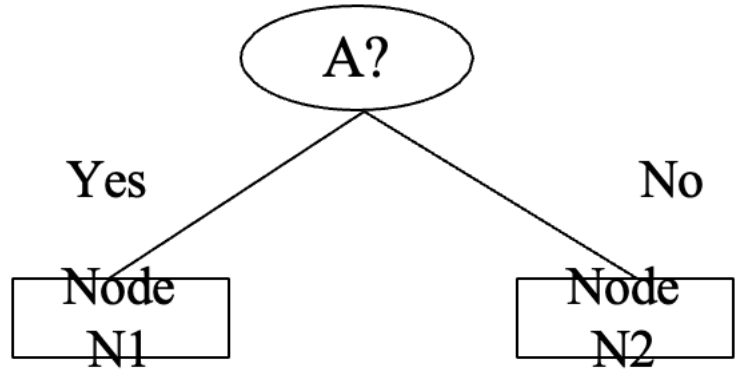
Gini(N2)
= 1 – $(4/7)^2$ – $(3/7)^2$
= 0.489

	N1	N2
C1	3	4
C2	0	3
Gini=0.342		

Gini(Children)
= 3/10 * 0
+ 7/10 * 0.489
= 0.342

Gini improves but error remains the same!!

Classification Error vs Gini Index



	Parent
C1	7
C2	3
Gini = 0.42	

	N1	N2
C1	3	4
C2	0	3
Gini=0.342		

	N1	N2
C1	3	4
C2	1	2
Gini=0.416		

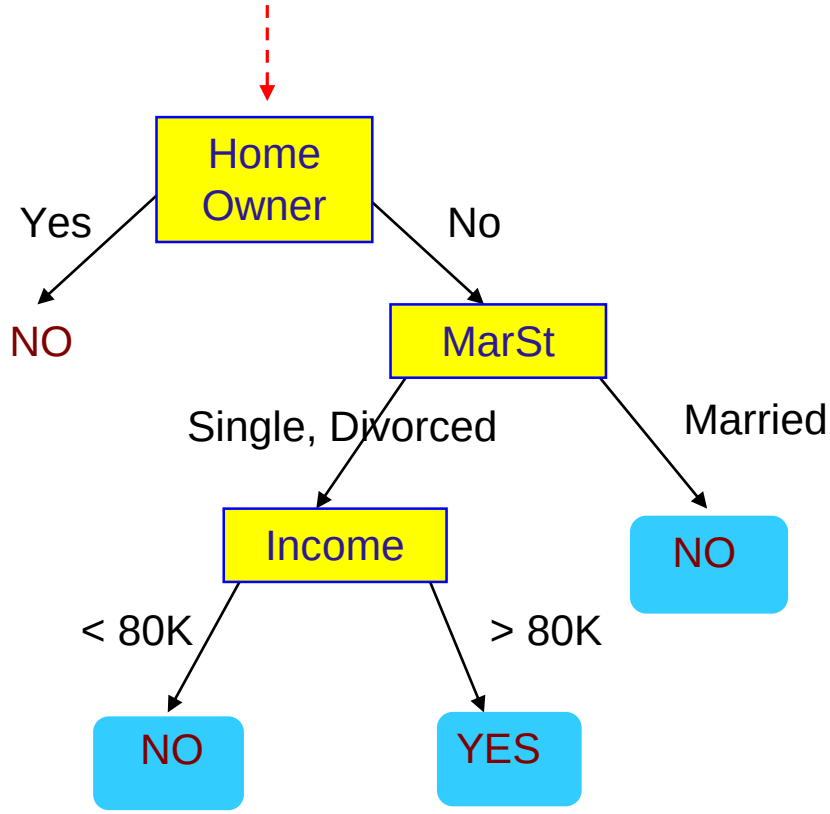
Misclassification error for all three cases = 0.3 !

Advantages & Disadvantages of Decision Trees

- Advantages:
  - Relatively inexpensive to construct
  - Extremely fast at classifying unknown records
  - Easy to interpret for small-sized trees
  - Robust to noise (especially when methods to avoid overfitting are employed)
  - Can easily handle redundant attributes
  - Can easily handle irrelevant attributes (unless the attributes are interacting)
- Disadvantages: .
  - Due to the greedy nature of splitting criterion, interacting attributes (that can distinguish between classes together but not individually) may be passed over in favor of other attributed that are less discriminating.
  - Each decision boundary involves only a single attribute

Apply Model to Test Data

Start from the root of tree.



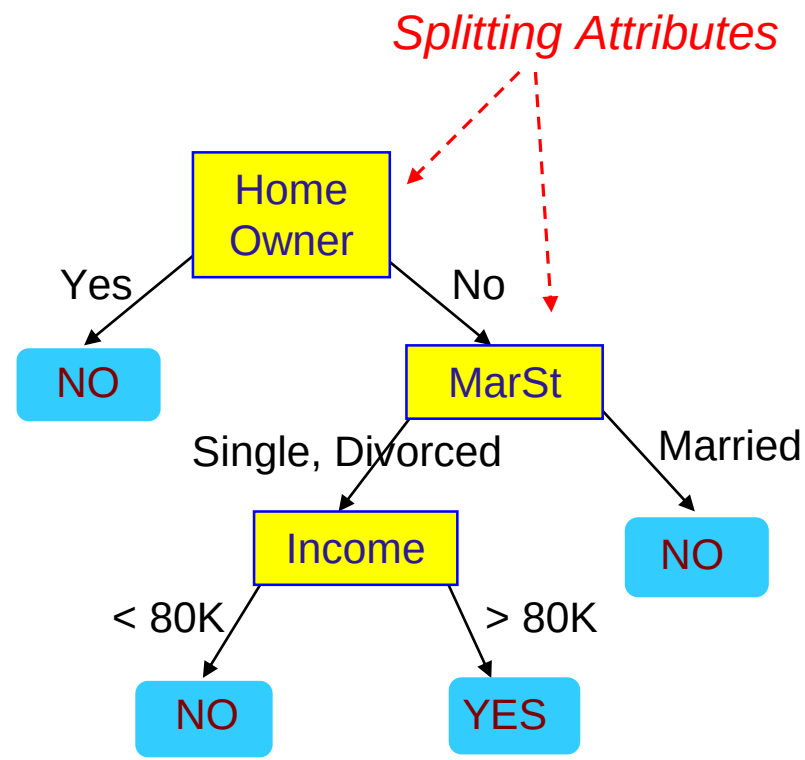
Test Data

Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?

Example of a Decision Tree

ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
	categorical	categorical	continuous	class
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

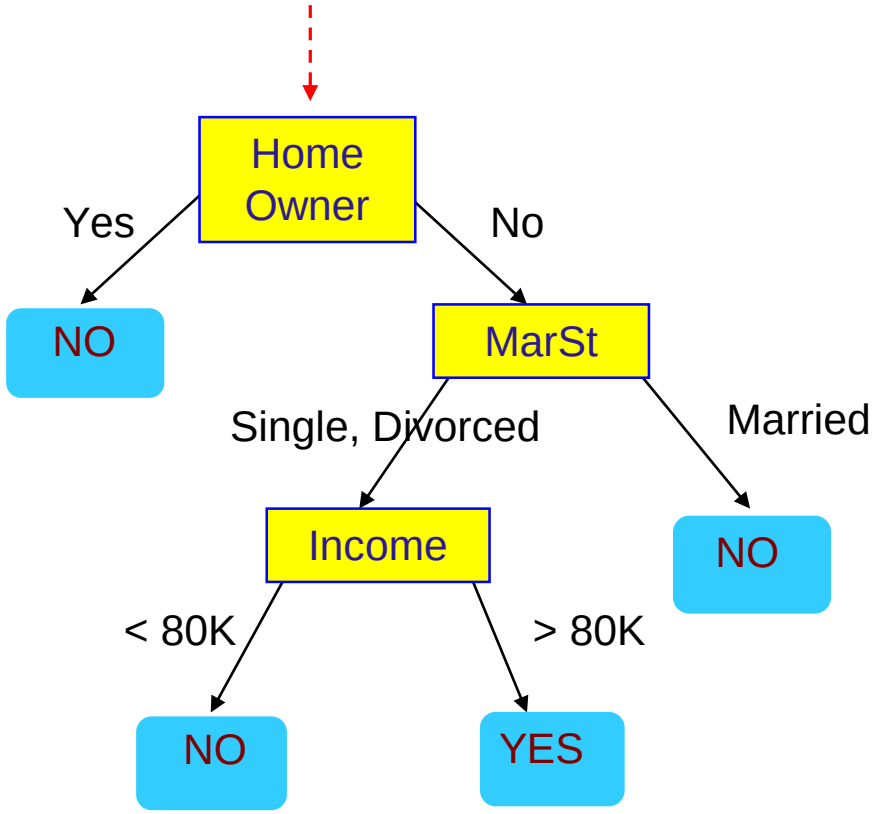
Training Data



Model: Decision Tree

Apply Model to Test Data

Start from the root of tree.



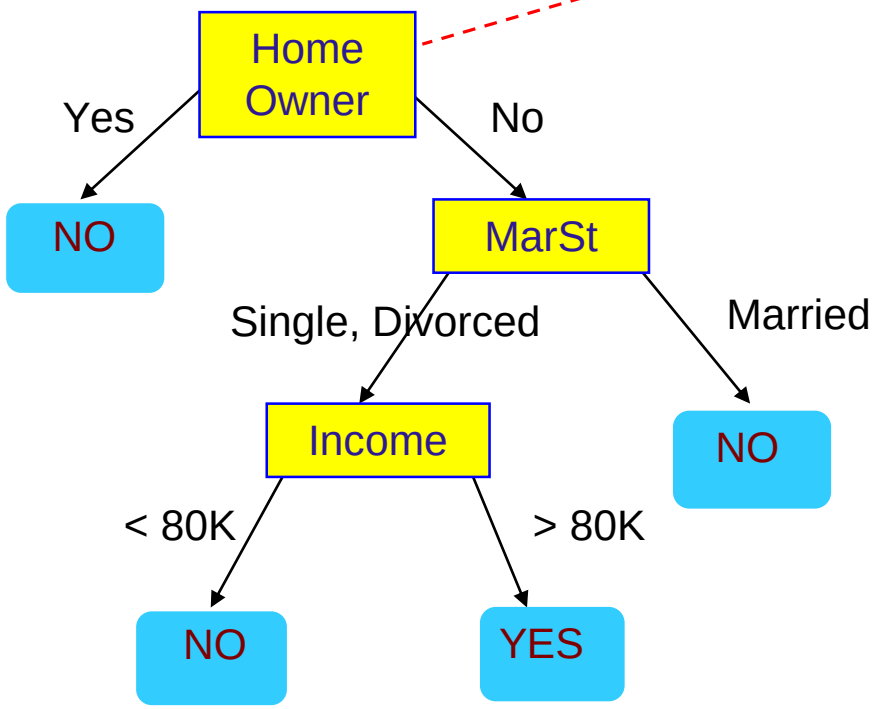
Test Data

Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?

Apply Model to Test Data

Test Data

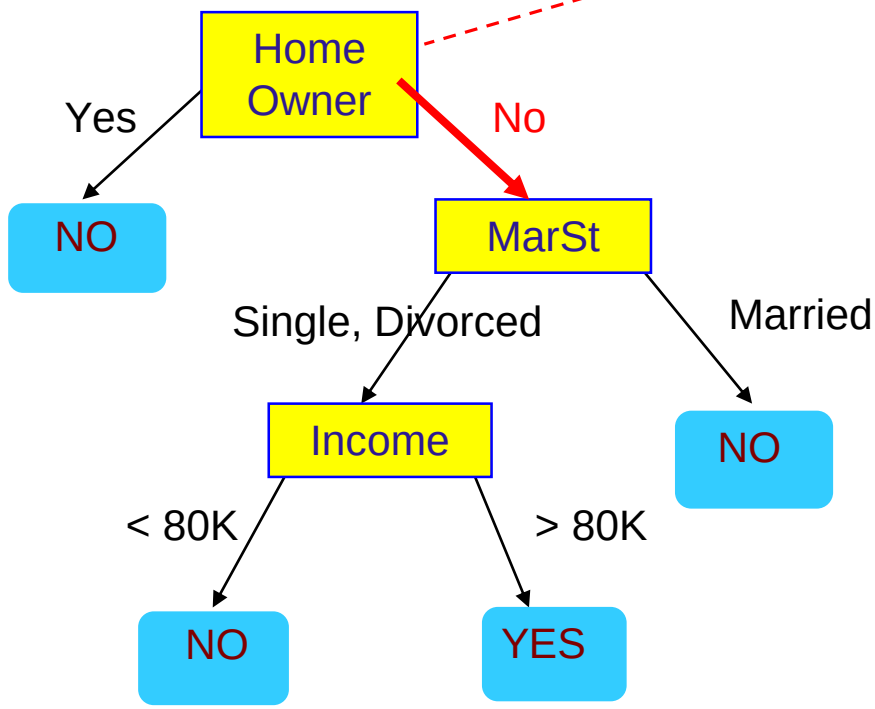
Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?



Apply Model to Test Data

Test Data

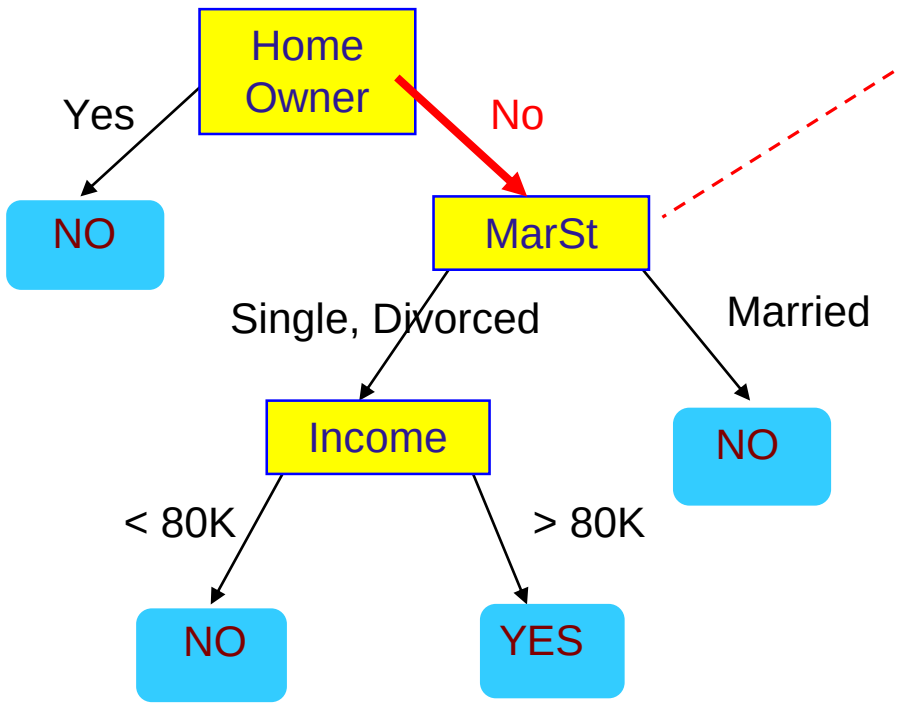
Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?



Apply Model to Test Data

Test Data

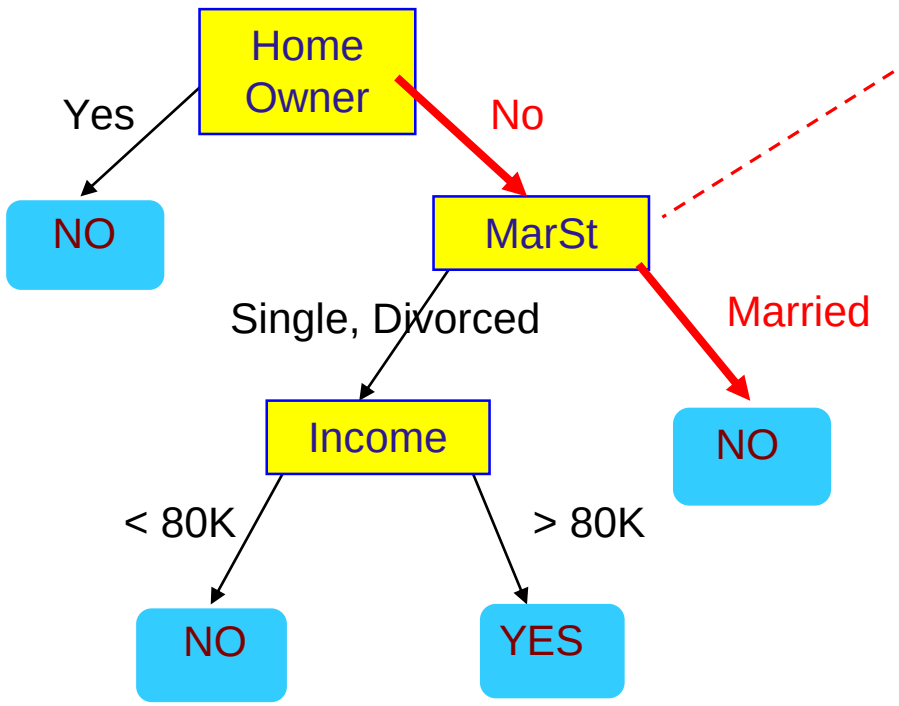
Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?



Apply Model to Test Data

Test Data

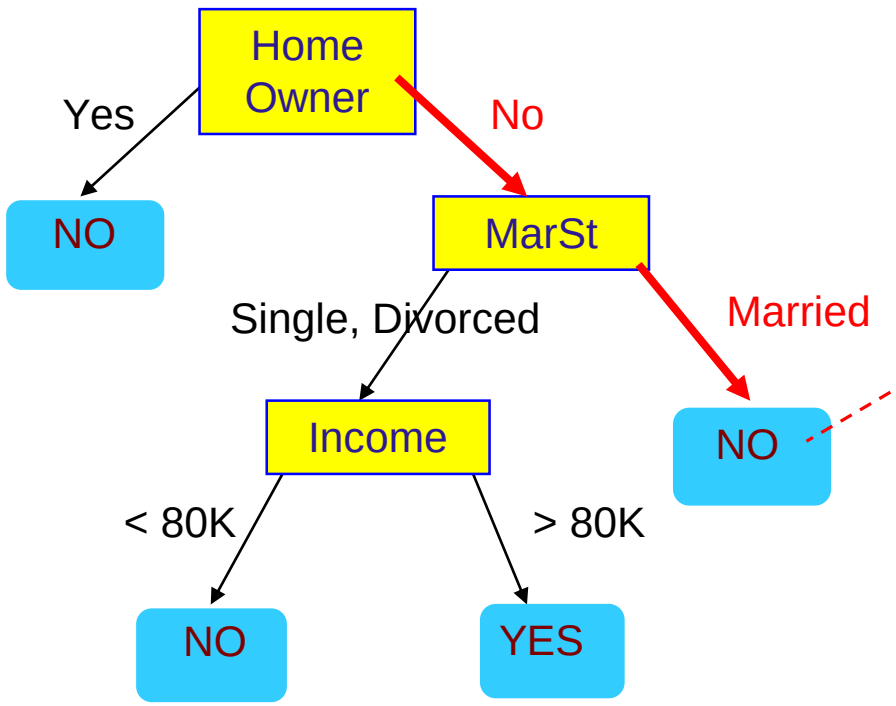
Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?



Apply Model to Test Data

Test Data

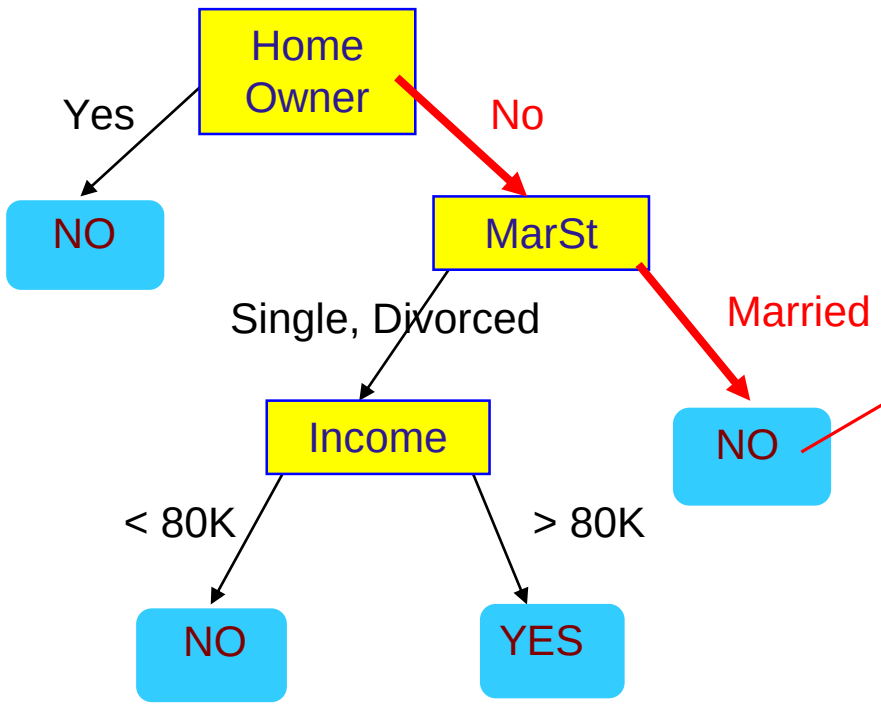
Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?



Apply Model to Test Data

Test Data

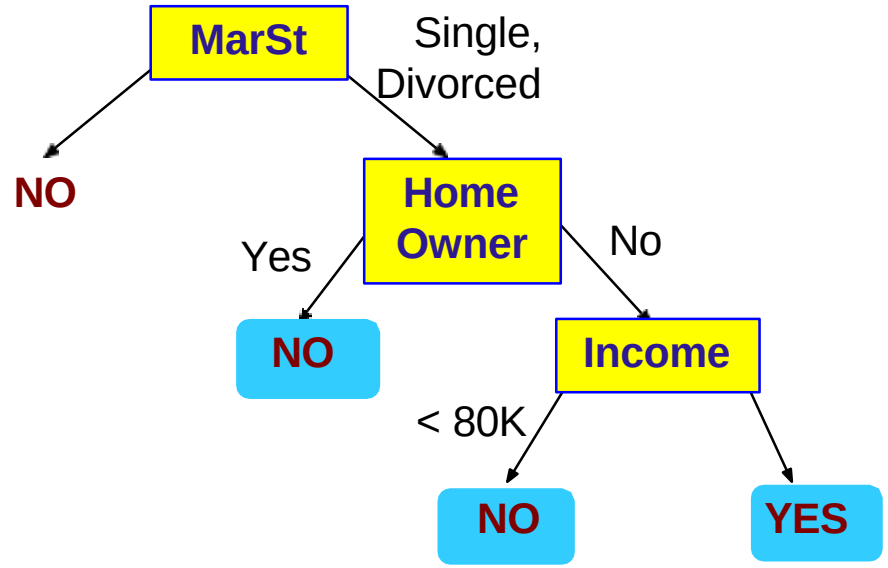
Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?



Assign Defaulted to "No"

Another example of Decision Tree

ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
	categorical	categorical	continuous	class
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes



There could be more than one tree that fits the same data!

When to Consider Decision Trees

- Instances are described by attribute–value pairs
- Target function is discrete valued
- Disjunctive hypothesis may be required
- Possibly noisy training data

Examples:

- Equipment or medical diagnosis
- Credit risk analysis
- Modeling calendar scheduling preferences

1.Basic Issues in D T Learning

1.1 Concept learning: An example

Given the data:

Day	Outlook	Temperature	Humidity	Wind	EnjoyTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

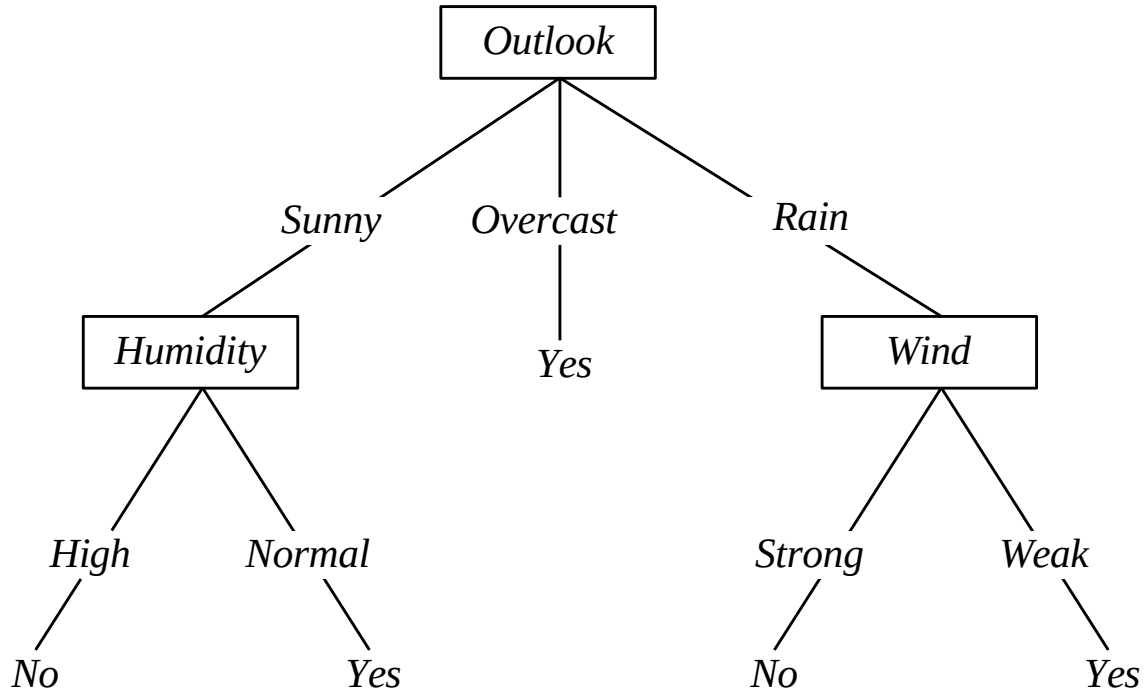
predict the value of EnjoyTennis for

$$\langle Outlook = sunny,\ Temp = cool,\ Humidity = high,\ Wind = strong \rangle$$

1.2. Decision tree representation

- Each internal node tests an attribute
- Each branch corresponds to attribute value
- Each leaf node assigns a classification

Example:
Decision Tree for *EnjoyTennis*



Another example:
A Tree to Predict C-Section Risk

Learned from medical records of 1000 women

Negative examples are C-sections

```
[833+,167-] .83+ .17-
Fetal_Presentation = 1: [822+,116-] .88+ .12-
| Previous_Csection = 0: [767+,81-] .90+ .10-
| | Primiparous = 0: [399+,13-] .97+ .03-
| | Primiparous = 1: [368+,68-] .84+ .16-
| | | Fetal_Distress = 0: [334+,47-] .88+ .12-
| | | | Birth_Weight < 3349: [201+,10.6-] .95+ .05-
| | | | Birth_Weight >= 3349: [133+,36.4-] .78+ .22-
| | | Fetal_Distress = 1: [34+,21-] .62+ .38-
| Previous_Csection = 1: [55+,35-] .61+ .39-
Fetal_Presentation = 2: [3+,29-] .11+ .89-
Fetal_Presentation = 3: [8+,22-] .27+ .73-
```

1.3.Top-Down Induction of Decision Trees

ID3 algorithm outline

[Ross Quinlan, 1979, 1986]

START

create the root *node*;
assign all examples to root;

Main loop:

1. *A* → the "best" decision attribute for next *node*;
2. for each value of *A*, create a new descendant of *node*;
3. sort training examples to leaf nodes;
4. if training examples perfectly classified, then STOP;
   else iterate over new leaf nodes

ID3 Algorithm: basic version

ID3(*Examples, Target_attribute, Attributes*)

- create a *Root* node for the tree; assign all *Examples* to *Root*;
- if all *Examples* are positive, return the single-node tree *Root*, with label=+;
- if all *Examples* are negative, return the single-node tree *Root*, with label=−;
- if *Attributes* is empty, return the single-node tree *Root*,
  with label = the most common value of *Target_attribute* in *Examples*;
- otherwise // Main loop:

  $A \rightarrow$ the attribute from *Attributes* that best* classifies *Examples*;
  the decision attribute for *Root* $\rightarrow A$;
  for each possible value v_i of A
  add a new tree branch below *Root*, corresponding to the test $A = v_i$;
  let $Examples_{v_i}$ be the subset of *Examples* that have the value v_i for A;
  if $Examples_{v_i}$ is empty
  below this new branch add a leaf node with label = the most common value of *Target attribute* in *Examples*;
  else
  below this new branch add the subtree
  ID3($Examples_{v_i}$, *Target attribute, Attributes*\{A\});
- return *Root*;

* The *best* attribute is the one with the highest *information gain*.

Entropy

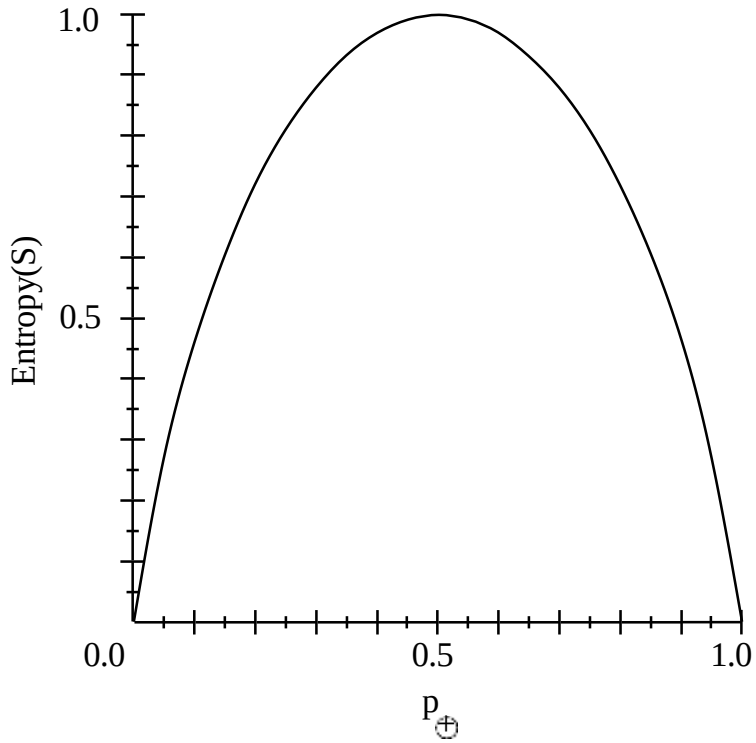
- Let S be a sample of training examples
  $p_{\oplus}$ is the proportion of positive examples in S
  $p_{\ominus}$ is the proportion of negative examples in S
- Entropy measures the impurity of S
- Information theory:

  $Entropy(S)$ = expected number of bits needed to encode $\oplus$ or $\ominus$ for a randomly drawn member of S (under the optimal, shortest-length code)

  The optimal length code for a message having the probability p is $-\log_2 p$ bits. So:

$$Entropy(S) = p_{\oplus}(-\log_2 p_{\oplus}) + p_{\ominus}(-\log_2 p_{\ominus}) = -p_{\oplus}\log_2 p_{\oplus} - p_{\ominus}\log_2 p_{\ominus}$$

Entropy



$$Entropy(S) = p_{\oplus}\log_2\frac{1}{p_{\oplus}} + p_{\ominus}\log_2\frac{1}{p_{\ominus}}$$

Note: By convention, $0\cdot\log_2 0 = 0$.

Statistical measures in D T learning: Entropy, and Information Gain

Information gain:

the expected reduction of the entropy of the instance set S due to sorting on the attribute A

$$Gain(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

Where:

- S is the original dataset
- A is the attribute
- S_v is the subset of S where attribute $A = v$

1.Basic Issues in D T Learning

1.1 Concept learning: An example

Given the data:

Day	Outlook	Temperature	Humidity	Wind	EnjoyTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

predict the value of EnjoyTennis for

$$\langle Outlook = sunny,\ Temp = cool,\ Humidity = high,\ Wind = strong \rangle$$

A partially learned tree

Attribute: Outlook

$Values\,(Outlook) = Sunny, Overcast, Rain$

$S = [9+, 5-]$ $\quad Entropy(S) = -\frac{9}{14}log_2\frac{9}{14} - \frac{5}{14}log_2\frac{5}{14} = 0.94$

$S_{Sunny} \leftarrow [2+, 3-]$ $\quad Entropy(S_{Sunny}) = -\frac{2}{5}log_2\frac{2}{5} - \frac{3}{5}log_2\frac{3}{5} = 0.971$

$S_{Overcast} \leftarrow [4+, 0-]$ $\quad Entropy(S_{Overcast}) = -\frac{4}{4}log_2\frac{4}{4} - \frac{0}{4}log_2\frac{0}{4} = 0$

$S_{Rain} \leftarrow [3+, 2-]$ $\quad Entropy(S_{Rain}) = -\frac{3}{5}log_2\frac{3}{5} - \frac{2}{5}log_2\frac{2}{5} = 0.971$

$$Gain\,(S, Outlook) = Entropy(S) - \sum_{v \in \{Sunny, Overcast, Rain\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S, Outlook)$$

$$= Entropy(S) - \frac{5}{14}Entropy(S_{Sunny}) - \frac{4}{14}Entropy(S_{Overcast})$$

$$- \frac{5}{14}Entropy(S_{Rain})$$

$$Gain(S, Outlook) = 0.94 - \frac{5}{14}0.971 - \frac{4}{14}\,0 - \frac{5}{14}0.971 = 0.2464$$

A partially learned tree

Attribute: Temp

$Values\ (Temp) = Hot, Mild, Cool$

$S = [9+, 5-]$ $\quad Entropy(S) = -\frac{9}{14} log_2 \frac{9}{14} - \frac{5}{14} log_2 \frac{5}{14} = 0.94$

$S_{Hot} \leftarrow [2+, 2-]$ $\quad Entropy(S_{Hot}) = -\frac{2}{4} log_2 \frac{2}{4} - \frac{2}{4} log_2 \frac{2}{4} = 1.0$

$S_{Mild} \leftarrow [4+, 2-]$ $\quad Entropy(S_{Mild}) = -\frac{4}{6} log_2 \frac{4}{6} - \frac{2}{6} log_2 \frac{2}{6} = 0.9183$

$S_{Cool} \leftarrow [3+, 1-]$ $\quad Entropy(S_{Cool}) = -\frac{3}{4} log_2 \frac{3}{4} - \frac{1}{4} log_2 \frac{1}{4} = 0.8113$

$$Gain\ (S, Temp) = Entropy(S) - \sum_{v \in \{Hot, Mild, Cool\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S, Temp)$$

$$= Entropy(S) - \frac{4}{14} Entropy(S_{Hot}) - \frac{6}{14} Entropy(S_{Mild})$$

$$- \frac{4}{14} Entropy(S_{Cool})$$

A partially learned tree

Attribute: Humidity

$Values\ (Humidity) = High, Normal$

$S = [9+, 5-]$ $\quad Entropy(S) = -\frac{9}{14} log_2 \frac{9}{14} - \frac{5}{14} log_2 \frac{5}{14} = 0.94$

$S_{High} \leftarrow [3+, 4-]$ $\quad Entropy(S_{High}) = -\frac{3}{7} log_2 \frac{3}{7} - \frac{4}{7} log_2 \frac{4}{7} = 0.9852$

$S_{Normal} \leftarrow [6+, 1-]$ $\quad Entropy(S_{Normal}) = -\frac{6}{7} log_2 \frac{6}{7} - \frac{1}{7} log_2 \frac{1}{7} = 0.5916$

$$Gain\ (S, Humidity) = Entropy(S) - \sum_{v \in \{High, Normal\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S, Humidity)$$

$$= Entropy(S) - \frac{7}{14} Entropy(S_{High}) - \frac{7}{14} Entropy(S_{Normal})$$

A partially learned tree

Attribute: Wind

$Values\ (Wind) = Strong, Weak$

$S = [9+, 5-]$ $\quad Entropy(S) = -\frac{9}{14} log_2 \frac{9}{14} - \frac{5}{14} log_2 \frac{5}{14} = 0.94$

$S_{Strong} \leftarrow [3+, 3-]$ $\quad Entropy(S_{Strong}) = 1.0$

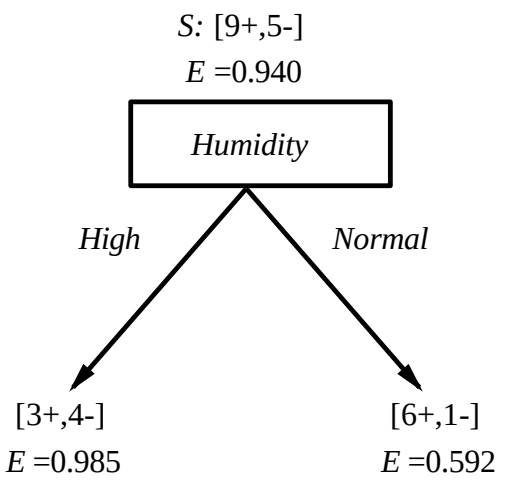
$S_{Weak} \leftarrow [6+, 2-]$ $\quad Entropy(S_{Weak}) = -\frac{6}{8} log_2 \frac{6}{8} - \frac{2}{8} log_2 \frac{2}{8} = 0.8113$

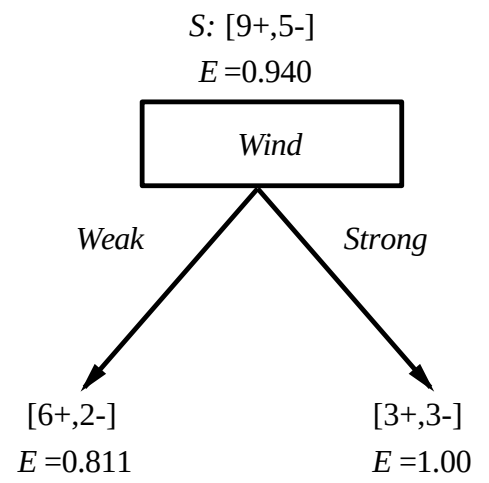
$$Gain\ (S, Wind) = Entropy(S) - \sum_{v \in \{Strong, Weak\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S, Wind) = Entropy(S) - \frac{6}{14} Entropy(S_{Strong}) - \frac{8}{14} Entropy(S_{Weak})$$

Selecting the Root Attribute

Which attribute is the best classifier?





Gain (S, Humidity)

= .940 - (7/14).985 - (7/14).592
= .151

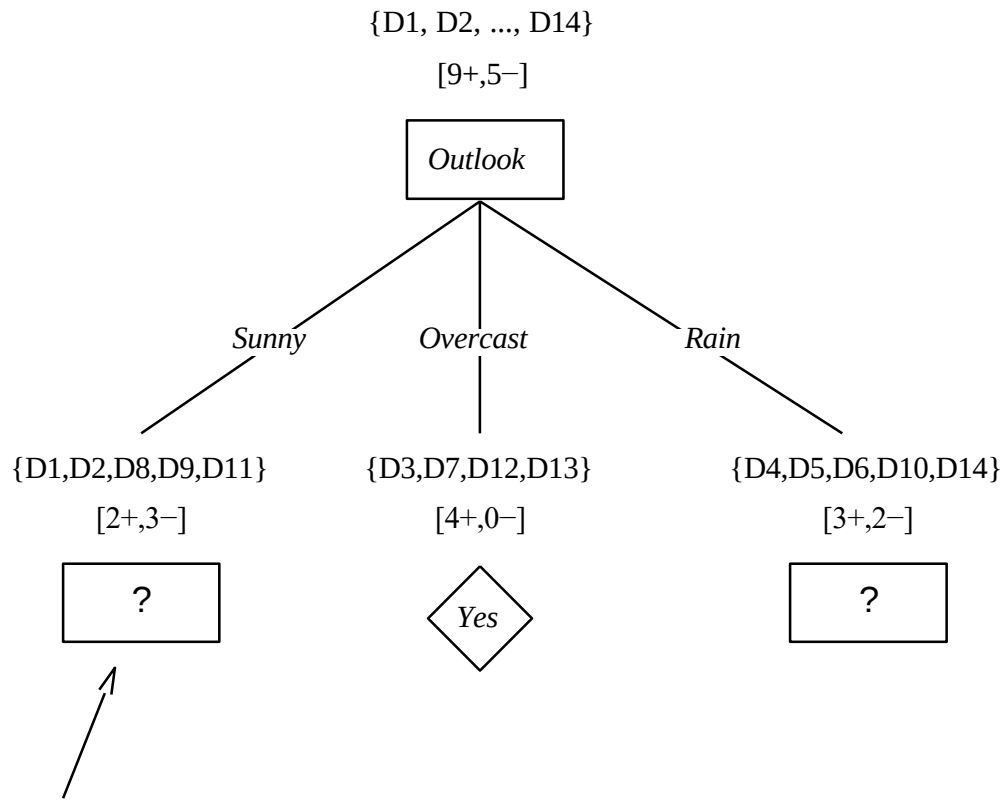
Gain (S, Wind)

= .940 - (8/14).811 - (6/14)1.0
= .048

Similarly,

$Gain(S, Outlook) = 0.246$
$Gain(S, Temperature) = 0.029$

A partially learned tree



Which attribute should be tested here?

S_{sunny} = {D1,D2,D8,D9,D11}

$Gain\ (S_{sunny}, Humidity) = .970 - (3/5)\ 0.0 - (2/5)\ 0.0 = .970$

$Gain\ (S_{sunny}, Temperature) = .970 - (2/5)\ 0.0 - (2/5)\ 1.0 - (1/5)\ 0.0 = .570$

$Gain\ (S_{sunny}, Wind) = .970 - (2/5)\ 1.0 - (3/5)\ .918 = .019$

A partially learned tree

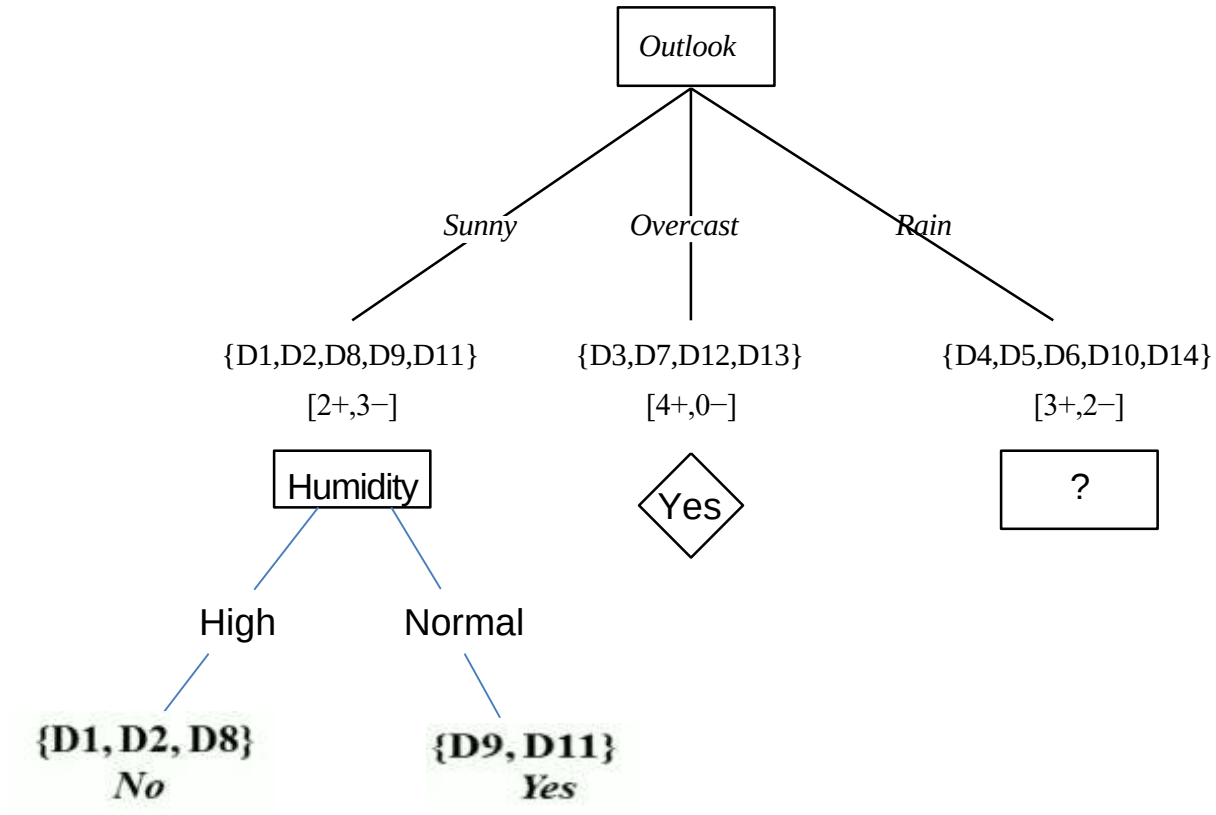
Day	Temp	Humidity	Wind	Play Tennis
D1	Hot	High	Weak	No
D2	Hot	High	Strong	No
D8	Mild	High	Weak	No
D9	Cool	Normal	Weak	Yes
D11	Mild	Normal	Strong	Yes

$$Gain(S_{sunny}, Temp) = 0.570$$

$$Gain(S_{sunny}, Humidity) = 0.97$$

$$Gain(S_{sunny}, Wind) = 0.0192$$

A partially learned tree



A partially learned tree

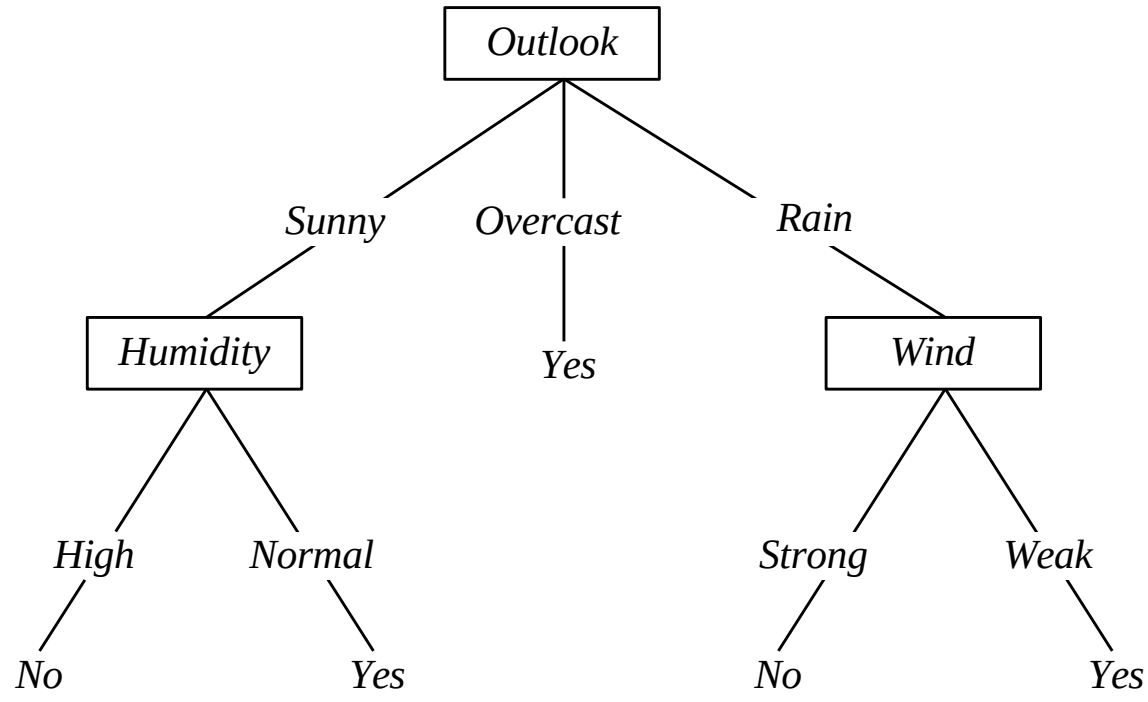
Day	Temp	Humidity	Wind	Play Tennis
D4	Mild	High	Weak	Yes
D5	Cool	Normal	Weak	Yes
D6	Cool	Normal	Strong	No
Dl0	Mild	Normal	Weak	Yes
Dl4	Mild	High	Strong	No

$$Gain(S_{Rain}, Temp) = 0.0192$$

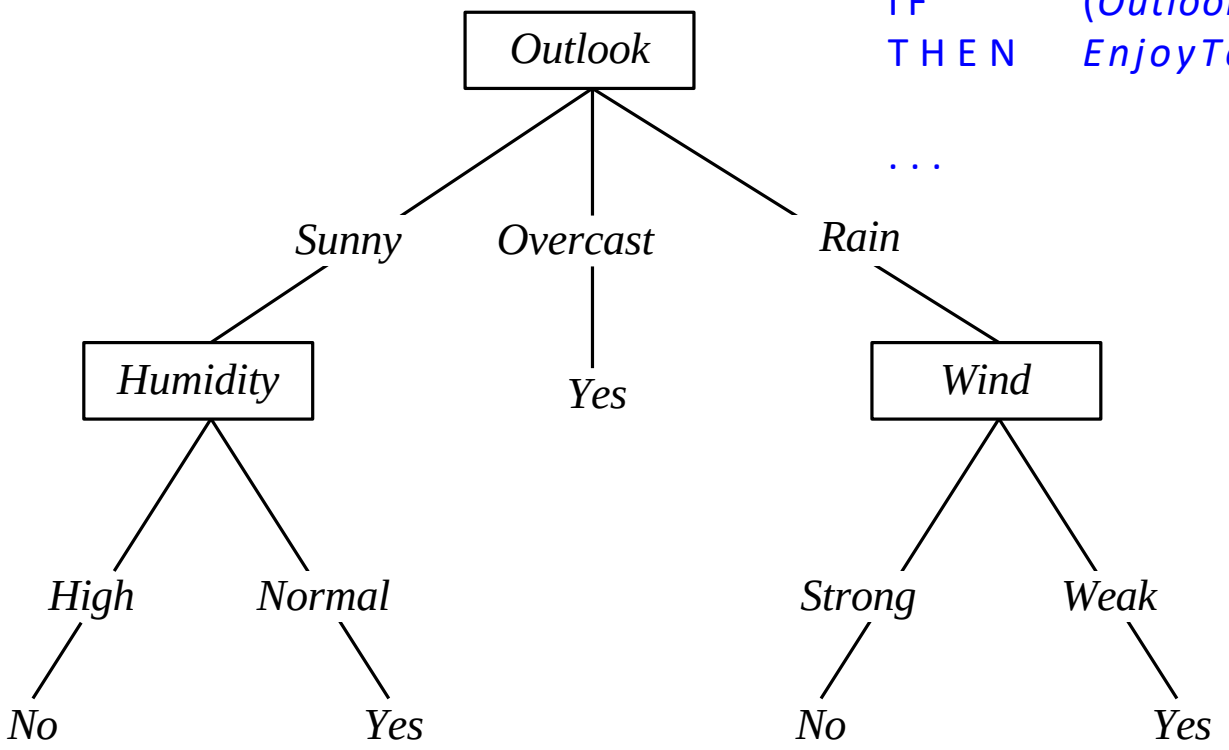
$$Gain(S_{Rain}, Humidity) = 0.0192$$

$$Gain(S_{Rain}, Wind) = 0.97$$

Final Tree


Outlook
Sunny
Overcast
Rain
Humidity
Yes
Wind
High
Normal
Strong
Weak
No
Yes
No
Yes

Converting A Tree to Rules



IF (*Outlook* = *Sunny*) ∧ (*Humidity* = *High*)
THEN *EnjoyTennis* = *No*

IF (*Outlook* = *Sunny*) ∧ (*Humidity* = *Normal*)
THEN *EnjoyTennis* = *Yes*

. . .

Hypothesis Space Search by ID3

- Hypothesis space is complete!

  The target function surely is in there...

- Outputs a single hypothesis

  Which one?

- Inductive bias:

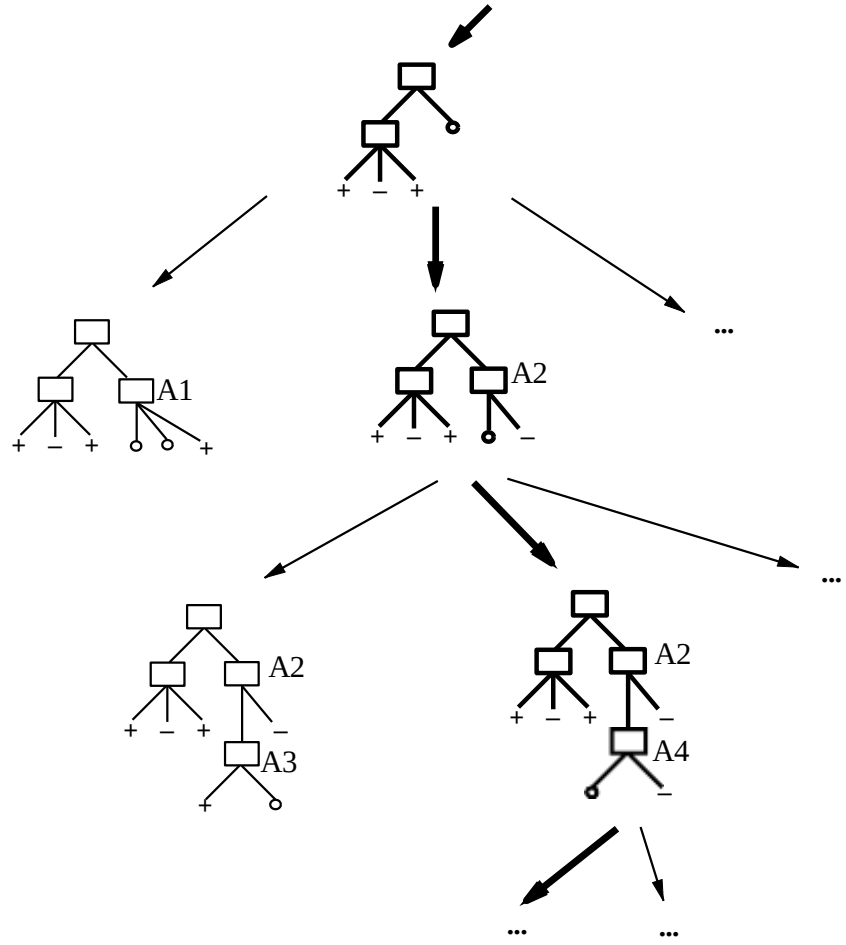
  approximate "prefer the shortest tree"

- Statisically-based search choices

  Robust to noisy data...

- No back-tracking

  Local minima...



Occam's Razor

- Prefer the simplest hypothesis that fits the data.

Why prefer short hypotheses?

Argument in favor:

- Fewer short hypotheses than long hypsotheses

→ a short hypothesis that fits data unlikely to be coincidence

→ a long hypothesis that fits data might be coincidence

What Makes It "Simple"?

"Simple" refers to:

Fewer nodes or branches in the tree.

Shallow depth (not too many levels).

Minimal assumptions—it doesn't overfit or memorize the data.

1.5 Complexity of decision tree induction

from "Data mining. Practical machine learning tools and techniques"

Witten et al, 3rd ed., 2011, pp. 199-200

- Input: d attributes, and m training instances
- Simplifying assumptions:

  (A1): the depth of the ID3 tree is $O(\log m)$,

  (i.e. it remains "bushy" and doesn't degenerate into long, stringy branches);

  (A2): [most] instances differ from each other;

  (A2'): the d attributes provide enough tests to allow the instances to be differentiated.
- Time complexity: $O(d\ m \log m)$.

Characteristics of Decision Tree Classifiers

- **Flexible & Nonparametric**: No assumptions about data distribution; handles both categorical and continuous data.
- **Multiclass Support**: Directly handles multiple classes—no need for binary decomposition.
- **Interpretable**: Easy to visualize and explain, especially when shallow.
- **Efficient**: Fast to train and test; uses greedy recursive partitioning.
- **Expressive**: Can represent complex discrete functions compactly.
- **Handles Missing Values**: Uses probabilistic, surrogate, or separate-class strategies.
- **Sensitive to Attribute Interactions**: May miss useful combinations due to greedy splits.

Credits

1. Data mining. Practical machine learning tools and techniques.
2. Adaptation of slides by Tom Mitchell for Machine Learning, CMU.
3. https://benchpartner.com/hunt-s-algorithm
4. https://blog.quantinsti.com/gini-index/